

Article

FinOps-Aware Budget-Constrained Optimization for Cloud Resource Management

Choong-Hee Cho 

Division of Computer Science and Engineering, Sahmyook University, Hwarangro 815,
Seoul 01795, Republic of Korea; cch@syu.ac.kr

Abstract

With the rise of Financial Operations (FinOps), cloud resource management requires the enforcement of strict budgetary guardrails rather than soft cost objectives. However, discrete Virtual Machine (VM) types often cause structural infeasibility, which existing methods fail to address. We formulate the Budget-Constrained VM Resizing problem under temporal hard constraints and establish the NP-hardness of the scalarized problem as a completeness result. To solve this, we propose the Budget-aware Dual (BD) solver, which utilizes a dual variable as a shadow price to dynamically steer candidate decisions toward budget feasibility without opaque penalty tuning. Extensive experiments demonstrate that BD significantly improves budget feasibility and operational stability compared to the baselines. In the run-rate setting, BD reduces candidate budget violations to zero once the budget enters feasible regimes at $\alpha \geq 0.6$ and substantially reduces operational churn, decreasing the change rate from 53.95% to 7.80% in an oscillatory workload scenario. BD also exhibits near-linear scalability and remains more than $100\times$ faster than NSGA-II at large fleet sizes. This framework provides a theoretically grounded and scalable approach for balancing economic efficiency, operational stability, and strict budget compliance.

Keywords: FinOps; cloud resource management; budget-constrained optimization; cloud cost management; hard budget constraints; dual-based optimization; multi-objective optimization; cloud computing

1. Introduction

Operating modern cloud services requires the solution of a broad set of optimization problems across multiple layers of the stack [1]. At the infrastructure layer, operators must provide and place resources, decide how to pack workloads onto instances, and react to changing demand through scheduling and scaling. At the application and Site Reliability Engineering (SRE) layers, they must maintain reliability targets while navigating transient load spikes, tail latency, and failures. At the business and operations layers, they must control spending, manage capacity commitments, and enforce organizational policies. These concerns are tightly coupled: a change that improves cost efficiency can degrade reliability, and an action that improves performance can trigger sustained spend increases. As a result, many cloud-operational tasks are naturally framed as online, multi-objective decision-making problems under uncertainty.

Within this landscape, Virtual Machine (VM) resizing is a fundamental operational mechanism [2]. Cloud providers offer a rich menu of VM specifications, and operators can adjust VM types as workloads evolve to reduce waste while maintaining sufficient headroom for tail demand. In large-scale VM fleets, however, resizing cannot be treated



Academic Editor: George Drosatos

Received: 24 February 2026

Revised: 11 March 2026

Accepted: 27 March 2026

Published: 29 March 2026

Copyright: © 2026 by the author.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

as a purely cost-minimization task. Changing VM specifications can incur migration or restart overheads and may degrade operational stability when performed frequently. Consequently, practical resizing policies must balance economic efficiency, tail-safety (e.g., overload risk), and operational stability under uncertainty, and they must do so in an online setting where decisions are made window by window.

With the growing adoption of Financial Operations (FinOps) practices, budgets are increasingly treated as operational guardrails rather than retrospective accounting targets [3]. Importantly, budgets in practice are often specified over time and appear in multiple, structurally different forms. Examples include a run-rate model that limits spending per decision window, an absolute cumulative model that caps total spending over a horizon, an incremental cumulative model that limits cumulative growth-driven expansions (e.g., scale-up cost increments), and a rolling model that constrains spending over a sliding window [4]. Under such temporal hard-budget policies, a resizing policy must produce actions that are consistent with an explicit feasibility notion aligned with deployment: feasible decisions are not merely those that reduce violations on average, but those that remain feasible (or minimize unavoidable infeasibility) under the active budget policy.

These requirements become more challenging when combined with a discrete VM-type pool [5]. Under tight budgets or abrupt workload shifts, there can exist decision windows in which no assignment from the discrete pool satisfies the active budget. We refer to this regime as structural infeasibility. Operationally, a window is structurally infeasible if even the lowest-cost assignment in the discrete VM-type pool violates the active budget constraint (i.e., the budget cannot be satisfied even after reducing all VMs to the cheapest available type). In this case, the objective cannot be to guarantee zero violations, because zero violations may be impossible; instead, the policy should minimize unavoidable violations while maintaining an acceptable sizing quality–stability trade-off. Moreover, real controllers do not execute a candidate decision directly. Candidate assignments must pass a shared compliance gate that checks the active temporal budget policy and produces a post-gate decision that can actually be deployed. If evaluation mixes candidate-level outcomes with post-processing outcomes, reported metrics such as cost saving and violation rate become ambiguous and comparisons across methods can be unfair.

Prior work on cloud resource management and VM resizing has explored a broad spectrum of approaches, ranging from heuristics and metaheuristics to reinforcement-learning-based policies [2,6,7]. However, when budgets are treated as first-class temporal hard constraints, several limitations remain. Existing methods often incorporate budgets indirectly through tuning-sensitive penalty terms, assume a single canonical budget form, or fail to clearly distinguish structurally infeasible regimes from algorithm-dependent behavior under discrete VM types. Moreover, method-specific post-processing or repair rules can alter the effective evaluation target, complicating fair comparisons across baselines.

In this paper, we address these gaps by formulating VM resizing as an online multi-objective optimization problem under temporal hard-budget constraints, where the objectives capture a sizing quality–stability trade-off. Sizing quality reflects both waste and tail-safety through overload-related terms, while stability captures resizing-induced changes. We study multiple temporal budget policies—run-rate, absolute cumulative, incremental cumulative, and rolling—and analyze how their structural differences induce distinct feasibility dynamics and trade-offs. To reflect operational deployment and eliminate ambiguity, we apply a shared compliance gate to the candidate decision produced by every method and compute all reported metrics from the resulting post-gate decisions. Unless stated otherwise, cost and stability metrics are computed from post-gate decisions; we additionally report candidate (pre-gate) feasibility diagnostics to assess budget internalization.

On top of this framework, we propose a Budget-aware Dual (BD) solver. BD does not rely on ad hoc penalty tuning to mitigate budget violations. Instead, it introduces a dual variable ν that represents budget pressure as a shadow price. When the budget is tight, ν increases and shifts decisions toward feasibility; when the budget is relaxed, ν decreases and the constraint becomes inactive. We provide theoretical intuition for the monotone behavior of ν with respect to the budget level and validate this behavior empirically across structurally different temporal budget formulations. This dual-variable perspective makes BD both effective and interpretable: its behavior can be understood through the lens of budget pressure rather than opaque heuristic rules.

To clarify the focus of our study, we investigate three closely related questions. First, we examine how different temporal hard-budget semantics—such as run-rate, absolute cumulative, incremental, and rolling budgets—affect feasibility dynamics and the trade-off between sizing quality and operational stability when the VM-type pool is discrete and structural infeasibility may arise. Second, we investigate whether a resizing policy guided by a single interpretable dual signal, denoted by ν and interpreted as a shadow price of budget pressure, can effectively internalize hard budget constraints while simultaneously suppressing excessive operational churn. Third, we analyze how evaluation methodology influences conclusions about budget feasibility and economic performance. In particular, we examine whether mixing candidate decisions with repaired post-processing outcomes obscures algorithmic behavior, and how applying a common compliance gate across all methods affects comparability and the interpretation of unavoidable violations caused by structural infeasibility. Together, these questions guide the formulation of the optimization framework, the design of the BD solver, and the experimental evaluation presented in this paper.

To address these questions, this work makes the following contributions:

- We define an operationally grounded problem setting for online VM resizing under temporal hard budgets with a discrete VM-type pool, explicitly modeling structural infeasibility. We also establish NP-hardness of the resulting scalarized problem to place BC-VMR within the known combinatorial complexity landscape.
- We unify four temporal budget models—per-window run-rate, absolute cumulative, rolling, and incremental cumulative—within a single framework, and empirically evaluate the proposed solver on three representative families: the per-window run-rate budget, the absolute cumulative budget, and the incremental cumulative budget. These three families are selected because they provide the clearest contrasts among local spending caps, horizon-level cumulative control, and growth-constrained upsizing, while the rolling-budget variant is used only as a supplementary diagnostic.
- We propose the Budget-aware Dual (BD) solver, which integrates temporal hard-budget constraints through an interpretable dual variable ν rather than tuning-dependent penalties or bespoke repair rules. We further formalize BD as an online projected dual-ascent method based on Lagrangian relaxation. For the run-rate budget case, which is stage-wise separable, we establish standard regret and violation bounds under a continuous relaxation and empirically quantify the relaxation-to-integer gap using a primal–dual lower bound.
- We conduct comprehensive simulator-based experiments using a behavior-driven synthetic workload generator and representative baselines (Static, Greedy, PRG, and NSGA-II), reporting operationally meaningful metrics such as cost saving, post-gate budget violation rate, and change rate, with auxiliary analyses of overload risk and operational churn where appropriate.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the problem setting, objectives, and temporal hard-budget constraints.

Section 4 discusses computational aspects of the formulation. Section 5 presents the BD solver and its theoretical properties. Section 6 presents and discusses experimental results, highlighting dual-variable behavior validation, comparisons across budget models, and performance under diverse workload regimes. Section 7 concludes and outlines future directions.

2. Related Work

2.1. Computational Hardness of Cloud Optimization

Cloud computing provides cost efficiency and elasticity; however, resource management at the data center scale—including VM placement, allocation, resizing, scheduling, load balancing, consolidation, and SLA/QoS enforcement—fundamentally reduces to combinatorial optimization problems, many of which have been repeatedly shown to be NP-hard or NP-complete [6,8–10]. VM placement and allocation naturally reduce to variants of the Bin Packing Problem, inheriting its computational intractability when minimizing the number of active servers for energy or cost efficiency [5,11,12]. In heterogeneous environments, these problems further reduce to the Generalized Assignment Problem or knapsack-type formulations with binary selection constraints, reinforcing NP-hardness [13–15]. VM resizing or right-sizing similarly involves selecting one configuration from a finite VM type set under multi-dimensional resource constraints and is commonly modeled as a multi-dimensional or multiple-choice knapsack problem [16,17]. Task scheduling, load balancing, and energy-aware consolidation inherit the hardness of classical scheduling, bin packing, and set cover formulations, particularly when extended to multi-resource and energy-cost-aware settings [18–20]. Provisioning under SLA/QoS constraints further connects to NP-hard combinatorial auction or social welfare maximization problems due to the need to select feasible combinations that jointly satisfy performance constraints [21–23]. Collectively, these results establish cloud optimization as a class of problems whose computational intractability is theoretically guaranteed, rendering exact solutions impractical at scale and motivating the widespread reliance on approximation, heuristic, metaheuristic, and learning-based methods for obtaining high-quality feasible solutions [24–27].

2.2. Cost-Aware Cloud Optimization

In cloud computing environments, resource management enables dynamic scalability by elastically provisioning resources in response to workload variations. Despite this flexibility, accurately predicting and optimizing resource usage remains a fundamental challenge due to workload uncertainty and system heterogeneity [28]. Given the NP-hardness reviewed in Section 2.1, cost has long been incorporated into cloud resource management objectives to reflect the economic nature of cloud services [10].

However, a consistent characteristic across cost-aware cloud optimization research is that cost is commonly treated as a soft objective rather than as a strict feasibility constraint [1,2,29,30]. In many formulations, cost appears as one component of a multi-objective function alongside performance or QoS metrics, allowing trade-offs through weighted combinations rather than enforcing explicit budget limits. This modeling approach implicitly assumes that cost can be relaxed in favor of performance gains, which limits its applicability in financially constrained operational settings.

A representative example of this paradigm is the widely adopted cost–performance trade-off model, often expressed in the form $\text{cost} + \alpha \text{ performance}$. Such formulations have been extensively applied to VM provisioning, task scheduling, and workflow optimization, where the weighting parameter α controls the relative importance of monetary cost versus performance objectives [31,32]. While this approach is effective in exploring Pareto-efficient

trade-offs for NP-hard problems, it relies on heuristic weight selection and provides no guarantee that the resulting solutions respect predefined budget boundaries.

Cost-aware considerations are also central to energy-aware VM consolidation, where reducing the number of active physical servers lowers energy consumption and operational expenses [33]. In these approaches, cost is typically derived from energy models and incorporated into the optimization objective together with performance or stability metrics. Nevertheless, even in this context, cost remains a secondary optimization target rather than a hard constraint, and budget violations may still occur under dynamic workload conditions.

This reliance on soft cost objectives exposes important limitations in practical cloud operations. Without explicit budget enforcement, optimization outcomes may exceed allowable spending, particularly in the presence of highly dynamic and unpredictable workloads [7,34]. From a FinOps perspective [3], which emphasizes cost accountability and continuous cost governance across engineering, finance, and business functions, existing cost-aware cloud optimization frameworks remain insufficient when budget compliance must be enforced. These limitations motivate optimization models that treat budget constraints as first-class, enforceable conditions rather than adjustable objectives.

2.3. Constraint Modeling in Cost-Aware Cloud Optimization

A smaller body of work models cost as an explicit feasibility constraint rather than as a soft objective. In early cost-aware formulations, multi-objective goals were commonly scalarized into weighted single-objective functions, which do not guarantee budget compliance when performance or availability objectives dominate [35,36].

To address this limitation, subsequent studies introduced single hard budget constraints that explicitly bound total cost over a fixed optimization horizon, thereby enforcing cost feasibility as a strict constraint rather than a tunable objective [36]. While such models improve budget compliance compared to soft-objective approaches, they are typically defined over a static horizon and fail to capture temporal spending dynamics arising from time-varying workloads, fluctuating demand, and complex pricing schemes. Consequently, fixed budget constraints may either be overly conservative or insufficient to regulate short-term spending spikes, motivating the development of time-indexed and history-aware budget constraint models.

2.4. Temporal Budget Constraints

Some studies further consider time-dependent hard feasibility constraints that impose feasibility requirements at multiple time points. Examples include per-window run-rate limits, cumulative budget trajectories, incremental limits and rolling-horizon constraints, and related online feasibility settings, which more closely reflect real-world FinOps practices [37–40].

A related line of research examines feasibility-driven online combinatorial scheduling under waiting-time constraints. Duque et al. [37] study online over-time processing, where tasks arrive dynamically, processing times are unknown, and the objective is to maximize the number of solved instances under waiting-time limits. Although that setting differs from cloud VM resizing under temporal budgets, it is relevant here because it shows how hard feasibility requirements can fundamentally reshape online optimization objectives. Our setting differs in the control variable (VM-type assignment rather than job sequencing), the feasibility semantics (temporal budget constraints rather than waiting-time constraints), and the deployment layer (candidate resizing decisions subsequently filtered through a compliance gate). Introducing time-dependent budget constraints changes the structure of cloud resource optimization in two distinct ways. First, some policies are time-indexed yet stage-wise separable, such as per-window run-rate caps, where feasibility is checked

independently in each window and does not depend on past spending. Second, temporally coupled policies—including cumulative budgets, rolling-horizon caps, and incremental limits—make feasibility at time t depend on the realized cost history. This coupling breaks the stage-wise separability exploited by many cost-aware heuristics and can render locally optimal decisions infeasible for future windows under history-dependent policies.

Many budget-aware approaches address this complexity by focusing on a single budget model at a time [39,40]. In VM provisioning and resizing, representative methods enforce feasibility using heuristic or rule-based mechanisms tied to a specific budget interpretation [38,41]. Similarly, workflow scheduling studies typically handle a single global budget by heuristically decomposing it into per-task or per-stage sub-budgets [39,42]. Although NP-hardness is often acknowledged, formal analysis is in many cases confined to simplified single-budget settings [38,39,41,42].

More recent work on cost-aware scheduling has introduced formal optimization models and NP-hardness results [43]. However, in many formulations, cost is modeled as a single-dimensional objective or constraint, without explicit support for heterogeneous or time-varying budget policies [38,39,41–43]. In practice, diverse budget controls—such as custom budget periods or spending alerts—are commonly enforced through external governance mechanisms rather than integrated into the optimization logic [4,44].

However, existing budget-aware formulations are often tied to a specific budgeting policy, making it difficult to reuse objectives and solvers across different temporal budget models. More broadly, prior work varies along several axes that are critical in FinOps-driven operations: whether budgets are enforced as hard feasibility constraints or only as soft objectives; what temporal semantics the budget expresses (run-rate, cumulative, rolling, incremental); how infeasibility is handled; whether a discrete VM-type pool induces structurally infeasible windows; whether operational stability is modeled; and whether evaluation distinguishes candidate decisions from deployable actions. To make these distinctions explicit and to position our contribution, we provide a structured comparison in Table 1. The concrete budget models used in this paper are formalized in Section 3.7, and the corresponding budget-constrained resizing problem and solver are presented in Sections 3–5 (with the algorithm detailed in Section 5).

Table 1. Structured comparison of representative budget-aware optimization approaches by budget semantics, feasibility/repair handling, stability modeling, evaluation protocol, and interpretability.

Work	Budget Model	Feasibility Handling	Stability	Evaluation Protocol	Interpretability
Thanasias et al. (2016) [38]	Budget +deadline constraint	Online Heuristic	No	Candidate-only	Low (rule-based heuristics)
Rizvi & Ramesh (2020) [39]	Single-horizon budget constraint	Heuristic decomposition	No	Candidate-only	Low (workflow heuristics)
Rajasekar & Santhiya (2024) [40]	Budget-constrained scheduling	Heuristic scheduling	No	Candidate-only	Low (heuristic scheduling rules)
Radhika & Sadasivam (2021) [41]	Budget-aware objective	Decision model	No	Candidate-only	Moderate (decision scoring model)

Table 1. *Cont.*

Work	Budget Model	Feasibility Handling	Stability	Evaluation Protocol	Interpretability
Duque et al. (2018) [37]	Waiting-time constraint	Online scheduling	Implicit	Candidate-only (online scheduling decisions)	Moderate (algorithmic scheduling model)
This work	Temporal hard budgets (run-rate, cumulative, rolling, incremental)	Dual-based control	Explicit	Candidate + post-gate	High (interpretable dual variable ν)

2.5. Comparative Positioning and Novelty

Table 1 highlights that most existing approaches treat budgets either as soft objectives or as single-horizon constraints and typically evaluate candidate decisions without distinguishing deployable actions. In contrast, BC-VMR explicitly models structural infeasibility induced by discrete VM-type pools and evaluates all methods under a common compliance gate aligned with deployment.

Importantly, the contribution of this work does not lie in introducing a new hardness archetype itself, since budget-constrained resource allocation naturally exhibits knapsack-like combinatorial structure. Instead, the novelty arises from the way several operational aspects are addressed together within a single formulation. In particular, this work simultaneously considers multiple temporal hard-budget semantics, explicitly models structural infeasibility caused by discrete VM-type pools, and evaluates resizing decisions through a deployment-aligned compliance gate that separates candidate and deployable actions. To the best of our knowledge, prior budget-aware cloud optimization studies do not integrate these operational considerations within a unified online VM resizing framework and evaluation protocol.

3. Problem Formulation

3.1. Overview and Notation

We consider a cloud platform that manages a fleet of N virtual machines (VMs). Time is discretized into T decision windows indexed by $t \in \{1, \dots, T\}$, each of fixed duration h hours. For example, a 5-min window corresponds to $h = \frac{1}{12}$. VMs are indexed by $i \in \{1, \dots, N\}$. At the beginning of each window t , the platform selects a VM type (instance specification) for each VM and keeps the selected type unchanged throughout the window. We consider two resource dimensions, CPU and memory. Let $\mathcal{R} := \{\text{cpu}, \text{mem}\}$ and index resources by $r \in \mathcal{R}$. For readability, Table 2 summarizes the main notation used throughout the paper.

Table 2. Summary of key notation used in the BC-VMR formulation.

Symbol	Meaning
ν	Dual variable (shadow price of budget pressure) used by BD.
α	Budget multiplier controlling budget strictness in experiments.
ρ	Stability weight in the scalarized objective (quality–stability trade-off).

Table 2. Cont.

Symbol	Meaning
λ	Overload–waste trade-off weight in the sizing-quality objective.
η	Dual update step size (learning rate).
v_{max}	Upper bound for v in the projected dual update.
w_{cpu}, w_{mem}	CPU and memory weights used when aggregating overload and waste.
h	Decision window length (hours).
T	Decision horizon length (number of windows).
N	Number of VMs in the fleet.
$\mathcal{K}$	Discrete VM-type set (instance-type pool).
$\mathbf{x}_t$	VM-type assignment vector at decision window t .
$\text{Cost}_t(\mathbf{x}_t)$	Total realized platform cost in decision window t .
$g_t(\cdot)$	Temporal hard-budget residual/feasibility function applied to the realized cost history up to time t .

At a high level, the formulation uses four window-level signals per VM: mean demand μ for typical load, tail demand q for burst protection, volatility σ for short-term variability, and entropy H for irregularity. These signals feed into a sizing-quality term that penalizes both waste and risk-weighted tail shortfall, while a separate stability term penalizes resizing events. The budget residual g_t then enforces the chosen temporal spending rule.

3.2. VM Type Set and Cost Model

Let $\mathcal{K}$ denote a finite set of VM types (instance types). Each type $k \in \mathcal{K}$ is characterized by its CPU capacity $\text{cap}_{\text{cpu}}(k)$, memory capacity $\text{cap}_{\text{mem}}(k)$, and hourly price $\text{price}(k)$. For $r \in \mathcal{R}$, we write $\text{cap}_r(k)$ to denote the capacity of resource r provided by type k . The maximum capacity over the VM-type pool is defined as

$$\text{cap}_r^{\max} := \max_{k \in \mathcal{K}} \text{cap}_r(k).$$

This quantity will be used as a normalization scale when required.

If a VM is assigned type k during a decision window of length h , the incurred cost in that window is

$$\text{cost}(k) := \text{price}(k) \cdot h.$$

Let $x_{i,t} \in \mathcal{K}$ denote the VM type assigned to VM i in window t , and let $\mathbf{x}_t := (x_{1,t}, \dots, x_{N,t})$ denote the assignment vector at time t . The total platform cost in window t is

$$\text{Cost}_t(\mathbf{x}_t) := \sum_{i=1}^N \text{cost}(x_{i,t}).$$

This explicit cost model is required because the budget constraints in Section 3.7 apply to actual spend rather than proxy utilization metrics.

3.3. Workload Observations as Window-Level Statistics

Resizing decisions must be robust to bursty and volatile workloads. Rather than summarizing the workload of a VM by a single average value, we represent each decision window by a small set of window-level statistics that capture both typical demand and tail variability.

For each VM i , resource $r \in \mathcal{R}$, and window t , let $d_{i,t,r}(l)$ denote the l -th observed resource demand sample within the window, for $l = 1, \dots, L$, measured in units consistent with $\text{cap}_r(\cdot)$. Under the experimental configuration in Table 3 (decision window is 20 min and sampling interval is 5 min), each window contains $L = 4$ demand samples, where each sample corresponds to an interval-aggregated demand value generated by the simulator. With $= 4$ samples per window, the empirical 0.95 quantile effectively behaves as a near-maximum tail proxy; we therefore use $q_{i,t,r}$ as a lightweight tail indicator rather than a high-precision percentile estimate. From these samples, the platform computes the following four statistics:

Table 3. Experimental configuration and hyperparameters.

Item	Symbols/Values	Notes
Decision window	$h = 20 \text{ min} (= \frac{1}{3} \text{ h})$	Fixed across experiments
Decision horizon	$T = 18$	Fixed across experiments
Sampling interval	5 min	Fixed across experiments
VM-type pool	$(c_k, m_k, p_k)_{k=1}^{15}$ where p_k is the per-hour price	Fixed finite VM type set
Fleet size (default)	$N = 50$	Used in Sections 6.3 and 6.4
Fleet size (scalability sweep)	$N \in \{50, 100, 200, 500, 1000, 10,000\}$	Used in Section 6.2.2
Base budget policy	B_0 calibrated once; held fixed	Calibrated using Static on a reference workload; fixed across runs/seeds
Budget strength levels	$\alpha \in \{0.1, 0.2, \dots, 1.0\}$ (full sweep)	Used in Sections 6.2–6.4
NSGA-II hyperparameters	$\text{pop} = 50; \text{gen} = 50; p_c = 0.7; p_m = 0.3$	Fixed across experiments; selection = NSGA-II; two-point crossover; uniform integer mutation
Dual update step size	$\eta = 1.0$	Fixed across experiments (Section 6.1.2)
Risk amplification weights	$\beta_1 = 0.3; \beta_2 = 0.4$	Fixed across all scenarios
Overload–waste trade-off	$\lambda = 0.5$	Fixed across experiments
Stability weight	$\rho = 10$	Used in Sections 6.3 and 6.4
Resource weights	$(w_{\text{cpu}}, w_{\text{mem}}) = (0.8, 0.2); w_{\text{cpu}} + w_{\text{mem}} = 1$	Implementation uses w_{cpu} and $(1 - w_{\text{cpu}})$ for memory weight.
Dual upper bound	$\nu_{\max} = 10^9$	Fixed across experiments

- Mean demand: $\mu_{i,t,r} := \frac{1}{L} \sum_{l=1}^L d_{i,t,r}(l)$
- Tail demand (p95): $q_{i,t,r} := \text{Quantile}_{0.95}(\{d_{i,t,r}(l)\}_{l=1}^L)$.
- Volatility (standard deviation): $\sigma_{i,t,r} := \sqrt{\frac{1}{L} \sum_{l=1}^L (d_{i,t,r}(l) - \mu_{i,t,r})^2}$
- Irregularity (entropy): For each VM i , window t , and resource r , we define the raw Shannon entropy as

$$H_{i,t,r}^{\text{raw}} := - \sum_{b=1}^{B_{\text{bin}}} p_{i,t,r}(b) \log_2 p_{i,t,r}(b)$$

where $b \in 1, \dots, B_{\text{bin}}$ indexes discretized demand bins and $p_{i,t,r}(b)$ denotes the empirical probability mass. In the experiments, we use $B_{\text{bin}} = 10$ equal-width bins over a

fixed absolute range $[0, \text{cap}_r^{\max}]$. The probabilities $p_{i,t,r}(b)$ are obtained by histogramming non-negative demand samples clipped to this range for entropy computation only. Overload beyond allocated capacity is handled separately by the tail/overload terms. We normalize entropy as

$$H_{i,t,r} := \frac{H_{i,t,r}^{\text{raw}}}{\log_2 B_{\text{bin}}}$$

so that $H_{i,t,r} \in [0, 1]$. Unless otherwise stated, $H_{i,t,r}$ is used in subsequent formulations (e.g., in $\omega(\sigma, H)$). Because each decision window contains only a small number of samples under our default monitoring granularity, $H_{i,t,r}$ is used here as a coarse irregularity descriptor rather than as a high-precision information-theoretic estimate. The fixed absolute range $[0, \text{cap}_r^{\max}]$ and the normalization by $\log_2 B_{\text{bin}}$ make entropy dimensionless, comparable across resources, and independent of the currently selected VM type. We use equal-width binning with $B_{\text{bin}} = 10$ to distinguish concentrated, dispersed, and bursty demand shapes without fitting a fine-grained density from a very small sample set.

Collectively, $(\mu_{i,t,r}, q_{i,t,r}, \sigma_{i,t,r}, H_{i,t,r})$ characterize baseline demand, tail behavior, intra-window variability, and unpredictability, respectively, and will be used to model the trade-off between cost efficiency and overload risk. At the beginning of window t , the platform observes statistics computed from the most recent completed window (i.e., window $t - 1$) and uses them to determine the assignment $\mathbf{x}_t$.

3.4. Decision Variables and Resizing Stability

In each decision window t , exactly one VM type is assigned to each VM:

$$x_{i,t} \in \mathcal{K}, \quad \forall i \in \{1, \dots, N\}, \forall t \in \{1, \dots, T\}$$

For notational convenience, we define $\text{cap}_r(x_{i,t}) := \text{cap}_r(k)$ where $k = x_{i,t}$.

Practical resizing systems often limit excessive configuration changes due to migration overhead and operational instability. To capture resizing stability, we define a change indicator

$$\text{chg}_{i,t} := \mathbb{I}\{x_{i,t} \neq x_{i,t-1}\}, \quad \forall i, \forall t \geq 2$$

where $\mathbb{I}\{\cdot\}$ denotes the indicator function (and we set $\text{chg}_{i,1} := 0$ by convention).

Optionally, a minimum dwell-time constraint can be imposed to prevent rapid oscillations. Let D denote the minimum number of consecutive windows that a VM must remain on a selected type before another change is allowed. This requirement can be written as

$$\sum_{\tau=t}^{t+D-1} \text{chg}_{i,\tau} \leq 1, \quad \forall i, \forall t \leq T - D + 1.$$

3.5. Modeling Over-Provisioning and Overload Risk

Resizing decisions must balance two competing goals: reducing waste due to over-provisioning and avoiding overload due to under-provisioning. To formalize this trade-off, we define resource-level overload and waste quantities for each VM, decision window, and resource dimension. We use the positive-part operator $[a]^+ := \max\{0, a\}$.

Throughout Sections 3.5 and 3.6, the statistics $\mu_{i,t,r}, q_{i,t,r}, \sigma_{i,t,r}$ and $H_{i,t,r}$ denote the most recent workload observations available at the decision time of window t , i.e., statistics computed from the immediately preceding completed window. This convention preserves an online decision-making interpretation without introducing additional indices.

3.5.1. Overload Risk (Under-Provisioning)

Overload risk arises when the allocated capacity is insufficient to accommodate high (tail) demand. We distinguish between a raw tail shortfall (a physical capacity violation at the tail) and a risk-weighted overload term, reflecting that the same shortfall is operationally more severe under highly volatile or irregular workloads.

For each VM i , window t , and resource r , we define the raw tail shortfall as

$$\delta_{i,t,r}(\mathbf{x}_t) := [q_{i,t,r} - \text{cap}_r(x_{i,t})]^+$$

where $q_{i,t,r}$ is the 95th-percentile demand and $\text{cap}_r(x_{i,t})$ is the allocated capacity. To modulate overload severity under volatile or irregular workloads, we introduce a workload-dependent weighting factor

$$\omega_{i,t,r} := \omega(\sigma_{i,t,r}, H_{i,t,r})$$

where $\sigma_{i,t,r}$ and $H_{i,t,r}$ denote the standard deviation and entropy of demand samples within window, respectively. We assume $\omega(\cdot)$ is nonnegative, monotone non-decreasing in each argument, and lower-bounded by 1, so that higher variability or irregularity yields a stronger penalty for overload risk. Importantly, $\omega_{i,t,r}$ is treated as a function of observed workload statistics only (i.e., it does not depend on the decision variable $x_{i,t}$). In the experiments, we instantiate $\omega_{i,t,r}$ using a bounded additive form (reported in Section 6.1), which guarantees $\omega_{i,t,r} \in [1, \omega_{\max}]$ for numerical stability while preserving monotone risk amplification.

The risk-weighted overload term is then defined as

$$\text{over}_{i,t,r}(\mathbf{x}_t) := \omega_{i,t,r} \cdot \delta_{i,t,r}(\mathbf{x}_t)$$

This construction penalizes assignments that provide insufficient headroom during bursts, while assigning larger penalties to the same tail shortfall when the workload exhibits higher variability or irregularity. The bounded additive form of ω is intentional. Volatility and irregularity are treated as two independent first-order signals that amplify the same raw tail shortfall, while the additive form preserves interpretability and avoids multiplicative blow-up when both signals are large. We therefore use ω as a lightweight, decision-independent risk amplifier rather than as a separate predictive model.

3.5.2. Waste (Over-Provisioning)

Conversely, waste occurs when the allocated capacity consistently exceeds typical (baseline) demand, leading to persistent unused resources. For each VM i , window t , and resource $r \in \mathcal{R}$, we define the waste amount as

$$\text{waste}_{i,t,r}(\mathbf{x}_t) := [\text{cap}_r(x_{i,t}) - \mu_{i,t,r}]^+.$$

where $\mu_{i,t,r}$ is the mean demand within window t . This term quantifies unused capacity relative to typical demand and encourages downsizing when allocated resources substantially exceed average needs.

3.5.3. Aggregated Overload and Waste Metrics

To facilitate objective formulation and system-level evaluation, we aggregate overload risk and waste across all VMs and resource dimensions. The total overload risk and waste in window t are defined as

$$\text{Over}_t(\mathbf{x}_t) := \sum_{i=1}^N \sum_{r \in \mathcal{R}} w_r \text{over}_{i,t,r}(\mathbf{x}_t),$$

$$\text{Waste}_t(\mathbf{x}_t) := \sum_{i=1}^N \sum_{r \in \mathcal{R}} w_r \text{waste}_{i,t,r}(\mathbf{x}_t).$$

The weights w_r reflect the relative importance of CPU and memory resources and are shared consistently across all objective terms. These weights are treated as tunable parameters that can be adjusted depending on the operational context or experimental setting, rather than being fixed constants. We normalize each resource by a fixed capacity scale $\text{cap}_r^{\max}$ so that CPU and memory contribute as dimensionless ratios. Because this is a constant scaling independent of the decision variables, it does not alter the combinatorial structure or the hardness results.

These aggregated quantities summarize system-wide tail-safety loss (risk-weighted shortfalls) and over-provisioning loss, respectively, and are used directly in the objective functions in Section 3.6.

3.6. Objective Functions

Based on the workload statistics and sizing metrics introduced in Sections 3.3–3.5, we formulate VM resizing as a multi-objective optimization problem that balances resource efficiency, tail safety, and operational stability under explicit budget constraints. For a planning horizon of T decision windows, the objectives and constraints are defined as follows:

$$f_1(\mathbf{x}_{1:T}) := \sum_{t=1}^T (\text{Waste}_t(\mathbf{x}_t) + \lambda \text{Over}_t(\mathbf{x}_t)), \quad (1)$$

$$f_2(\mathbf{x}_{1:T}) := \sum_{t=2}^T \sum_{i=1}^N \text{chg}_{i,t}, \quad (2)$$

$$\text{s. t. } x_{i,t} \in \mathcal{K}, \forall i, t \quad (3)$$

$$g_t(\text{Cost}_1(\mathbf{x}_1), \dots, \text{Cost}_t(\mathbf{x}_t)) \leq 0, \quad \forall t \quad (4)$$

The first objective f_1 captures overall sizing quality by aggregating, over time, the trade-off between over-provisioning and under-provisioning. The term $\text{Waste}_t(\mathbf{x}_t)$ measures unused capacity relative to mean demand, while $\text{Over}_t(\mathbf{x}_t)$ measures risk-weighted tail shortfalls based on p95 demand, as defined in Section 3.5. Because overload risk is already weighted at the VM–resource level according to workload volatility and irregularity, the same physical shortfall incurs a larger penalty for bursty or unpredictable workloads. The parameter $\lambda \geq 0$ controls the relative importance of tail safety versus resource efficiency and is treated as a tunable parameter that can be adjusted depending on operational priorities or experimental scenarios. Although cloud pricing is explicitly defined in Section 3.2, monetary cost is not directly minimized in the objective function. Instead, cost is enforced as a hard feasibility constraint through the budget models in Section 3.7, while the objective function focuses on balancing resource waste and overload risk within the feasible budget region. This separation reflects a FinOps-oriented design, where budget compliance is mandatory and performance–efficiency trade-offs are optimized under that constraint.

The second objective f_2 captures operational stability by penalizing frequent resizing actions. The indicator $\text{chg}_{i,t}$, introduced in Section 3.4, equals one when VM i changes its assigned type between consecutive windows and zero otherwise. By minimizing f_2 , the formulation discourages excessive configuration changes that may incur migration overhead or operational disruption. This formulation intentionally penalizes the occurrence of resizing events rather than their magnitude, reflecting the operational risk and coordination overhead associated with any configuration change, irrespective of its size.

The budget constraint is expressed in a generic form through the function $g_t(\cdot)$, which enforces feasibility over the realized platform cost sequence $\{\text{Cost}_t(\mathbf{x}_t)\}$. This

abstraction allows the same objective structure to accommodate different operational budgeting policies. In Section 3.7, the constraint $g_t(\cdot)$ is instantiated to represent concrete scenarios such as per-window run-rate limits, absolute cumulative budgets, incremental growth constraints, and rolling-horizon budgets.

Overall, the proposed formulation yields a clear Pareto trade-off between sizing quality and operational stability, while explicitly enforcing budget feasibility through constraints rather than objectives. This separation enables flexible FinOps-aware policy modeling without altering the core optimization structure.

3.7. Budget Constraints

Cloud cost control in practice is governed by externally imposed FinOps policies, which must be satisfied as hard feasibility requirements rather than soft optimization preferences. In our formulation, budget constraints are imposed directly on the realized platform cost, not on proxy metrics such as utilization or allocated capacity. Specifically, the total cost incurred in window t is given by $\text{Cost}_t(\mathbf{x}_t)$, as defined in Section 3.2, and all budget models operate on the sequence $\{\text{Cost}_1(\mathbf{x}_1), \dots, \text{Cost}_t(\mathbf{x}_t)\}$.

To accommodate diverse real-world budgeting policies within a unified framework, we express budget feasibility through a generic constraint function

$$g_t(\text{Cost}_1(\mathbf{x}_1), \dots, \text{Cost}_t(\mathbf{x}_t)) \leq 0, \quad \forall t \in \{1, \dots, T\}$$

which corresponds to constraint (4) in Section 3.6. For notational convenience, we equivalently write

$$g_t(\mathbf{x}_{1:t}) := g_t(\text{Cost}_1(\mathbf{x}_1), \dots, \text{Cost}_t(\mathbf{x}_t))$$

since the realized cost sequence up to time t is uniquely determined by $\mathbf{x}_{1:t}$. Different operational budget policies are obtained by instantiating $g_t(\cdot)$ appropriately. This abstraction allows the objective structure to remain unchanged while enabling flexible modeling of short-term rate limits, long-term cumulative caps, growth constraints, and rolling-horizon budgets. All budget parameters are assumed nonnegative for all t .

These four models represent, respectively, (i) instantaneous per-window caps, (ii) long-horizon cumulative caps, (iii) limits on abrupt cost increases, and (iv) sliding-horizon caps over recent windows.

3.7.1. Run-Rate (Per-Window) Budget

The run-rate budget enforces a strict upper bound on the cost incurred in each individual decision window. Let $B_{\text{rate_pw}}(t)$ denote the maximum allowed cost in window t . The corresponding constraint is

$$\text{Cost}_t(\mathbf{x}_t) \leq B_{\text{rate_pw}}(t), \quad \forall t,$$

or equivalently,

$$g_t^{\text{rate_pw}} = \text{Cost}_t(\mathbf{x}_t) - B_{\text{rate_pw}}(t) \leq 0.$$

This budget model directly limits instantaneous spending and prevents short-term cost spikes. Because feasibility must be satisfied independently in every window, no borrowing of budget across time is allowed. As a result, the run-rate budget is conservative under bursty workloads but aligns well with operational settings where maintaining a stable burn rate is essential.

From an analytical perspective, the run-rate formulation also has an important structural property. The run-rate budget model also enables a tractable analytical formulation because the budget constraint is applied independently at each decision window. This

separability allows the discrete VM assignment problem to be relaxed into a continuous formulation, which enables the primal–dual analysis developed in Section 5. In contrast, cumulative, rolling, and incremental budget models introduce inter-temporal coupling across windows, which significantly complicates theoretical analysis. For these budget semantics, the BD solver is therefore evaluated primarily through empirical experiments.

3.7.2. Absolute Cumulative Budget

The absolute cumulative budget constrains the total cost accumulated up to each decision window. Let $B_{\text{abs_cum}}(t)$ denote the maximum allowable cumulative cost by the end of window t . The constraint is expressed as

$$\sum_{\tau=1}^t \text{Cost}_{\tau}(\mathbf{x}_{\tau}) \leq B_{\text{abs_cum}}(t), \quad \forall t,$$

or equivalently,

$$g_t^{\text{abs_cum}} = \sum_{\tau=1}^t \text{Cost}_{\tau}(\mathbf{x}_{\tau}) - B_{\text{abs_cum}}(t) \leq 0.$$

Unlike the run-rate model, the cumulative budget allows temporary overspending as long as it is compensated by lower spending in other windows. This formulation naturally captures common FinOps practices such as daily, weekly, or monthly budget caps, and it supports arbitrary cumulative budget trajectories without altering the optimization structure.

3.7.3. Incremental Budget (Upsizing-Driven Cost Cap)

The incremental budget limits how much the platform is allowed to increase spend via scale-up (upsizing) actions between consecutive decision windows. Unlike absolute cumulative budgets, this model constrains only upsizing-driven cost increases on a per-window basis, without limiting total accumulated spend. Let $B_{\text{inc_cum}}(t)$ denote the per-window cap on upsizing-driven cost increments at window t . We measure the net platform cost change between consecutive windows as $\text{Cost}_t(\mathbf{x}_t) - \text{Cost}_{t-1}(\mathbf{x}_{t-1})$ and charge only its positive part, i.e., $[\text{Cost}_t(\mathbf{x}_t) - \text{Cost}_{t-1}(\mathbf{x}_{t-1})]^+$. Hence, downsizing-induced savings do not create budget credit: when $\text{Cost}_t(\mathbf{x}_t) - \text{Cost}_{t-1}(\mathbf{x}_{t-1}) < 0$, the charged increment is zero. Since $B_{\text{inc_cum}}(t) \geq 0$, this positive-part charging can be enforced by the affine constraint below. For $t \geq 2$, the constraint is given by

$$\text{Cost}_t(\mathbf{x}_t) - \text{Cost}_{t-1}(\mathbf{x}_{t-1}) \leq B_{\text{inc_cum}}(t)$$

which can be written as

$$g_t^{\text{inc}} := \text{Cost}_t(\mathbf{x}_t) - \text{Cost}_{t-1}(\mathbf{x}_{t-1}) - B_{\text{inc_cum}}(t) \leq 0$$

with $g_1^{\text{inc}} = 0$ by convention. For $t = 1$, we treat $\mathbf{x}_1$ as given; alternatively, a run-rate cap $\text{Cost}_1(\mathbf{x}_1) \leq B_{\text{rate_pw}}(1)$ may be applied.

Although indexed cumulatively, feasibility is evaluated window-wise; thus, the constraint functions as a per-window cap on upsizing-driven cost increases rather than as a cumulative spending limit. This model enforces financial smoothness by limiting abrupt cost escalations while allowing downsizing actions. The incremental budget therefore complements the resizing-stability objective f_2 , which penalizes the frequency of configuration changes but does not directly constrain their financial impact. Since $B_{\text{inc_cum}}(t) \geq 0$, the affine constraint provides an exact representation of the positive-part cap.

We use this affine form as the budget residual g_t^{inc} in the BD solver to preserve the per-window separability. This window-wise formulation is intentional, as it enables the incremental budget to be integrated into the BD solver without breaking the stage-wise structure required for scalable online optimization.

3.7.4. Rolling Budget

The rolling budget constrains spending over a sliding horizon of fixed length. Let $W \in \mathbb{Z}_+$ denote the rolling-horizon length in the number of decision windows, and let $B_{\text{roll}}(t)$ denote the maximum allowed cost over the most recent W windows ending at t . The constraint is written as

$$\sum_{\tau=\max\{1, t-W+1\}}^t \text{Cost}_{\tau}(\mathbf{x}_{\tau}) \leq B_{\text{roll}}(t),$$

or equivalently,

$$g_t^{\text{roll}} := \sum_{\tau=\max\{1, t-W+1\}}^t \text{Cost}_{\tau}(\mathbf{x}_{\tau}) - B_{\text{roll}}(t) \leq 0.$$

The rolling budget lies between per-window and fully cumulative constraints. It allows short-lived cost bursts while preventing sustained overspending over any recent horizon of length W . This formulation closely reflects continuous budget monitoring practices and is well suited to rolling-window FinOps policies.

3.8. Budget-Constrained VM Resizing Problem

We are now ready to formally define the budget-constrained VM resizing problem. Over a planning horizon of T decision windows, the platform selects, at each window t , a VM type assignment vector $\mathbf{x}_t = (x_{1,t}, \dots, x_{N,t}) \in \mathcal{K}^N$, where $x_{i,t}$ denotes the VM type assigned to VM i during window t . Let $\mathbf{x}_{1:T} := (\mathbf{x}_1, \dots, \mathbf{x}_T)$ denote the full resizing plan. Budget feasibility is enforced through the constraint family

$$g_t(\mathbf{x}_{1:t}) \leq 0, \quad \forall t \in \{1, \dots, T\} \quad (5)$$

where $\text{Cost}_t(\mathbf{x}_t)$ is the realized platform cost in window t , and $g_t(\cdot)$ is instantiated by one of the budget models introduced in Section 3.7 (run-rate, absolute cumulative, incremental, or rolling). The feasible set is therefore defined as

$$\mathcal{F} := \{\mathbf{x}_{1:T} | x_{i,t} \in \mathcal{K} \forall i, t, g_t(\mathbf{x}_{1:t}) \leq 0 \forall t\}, \quad (6)$$

optionally augmented with operational constraints such as minimum dwell time when required.

3.8.1. Bi-Objective Formulation

Using the objective functions defined in Section 3.6, we formulate the Budget-Constrained VM Resizing problem, hereafter referred to as BC-VMR, as the following bi-objective optimization:

$$\min_{\mathbf{x}_{1:T} \in \mathcal{F}} (f_1(\mathbf{x}_{1:T}), |f_2(\mathbf{x}_{1:T})) \quad (7)$$

where f_1 aggregates, over time, the trade-off between resource waste and risk-weighted overload, and f_2 penalizes resizing frequency to capture operational stability. A feasible solution $\mathbf{x}_{1:T}^* \in \mathcal{F}$ is Pareto optimal if no other feasible sequence weakly improves both objectives and strictly improves at least one. The Pareto frontier thus character-

izes the fundamental trade-off between sizing quality and resizing stability under hard budget compliance.

3.8.2. Scalarized Family

While the problem is inherently bi-objective, concrete operating points are often required for algorithm design and experimental evaluation. We therefore introduce a scalarized variant of *BC-VMR*, denoted by *BC-VMR*_ρ for $\rho \geq 0$:

$$\min_{\mathbf{x}_{1:T} \in \mathcal{F}} F_{\rho}(\mathbf{x}_{1:T}), \quad F_{\rho}(\mathbf{x}_{1:T}) = f_1(\mathbf{x}_{1:T}) + \rho f_2(\mathbf{x}_{1:T}) \quad (8)$$

Varying ρ selects different Pareto-efficient operating regimes, which is useful both for practical deployment and for comparative evaluation using metrics such as hypervolume.

3.8.3. Stage-Wise Structure and Per-Window Interpretation

The scalarized objective admits an additive stage-wise representation:

$$F_{\rho}(\mathbf{x}_{1:T}) := \sum_{t=1}^T (\text{Waste}_t(\mathbf{x}_t) + \lambda \text{Over}_t(\mathbf{x}_t)) + \rho \sum_{t=2}^T \sum_{i=1}^N \mathbb{I}\{x_{i,t} \neq x_{i,t-1}\}, \quad (9)$$

where each per-window cost depends on the current assignment $\mathbf{x}_t$ and, through the change indicator, on the previous assignment $\mathbf{x}_{t-1}$. Moreover, as shown in Section 3.5, $\text{Waste}_t(\cdot)$ and $\text{Over}_t(\cdot)$ decompose across VMs and resource dimensions, revealing a per-VM separable structure within each window. This representation does not imply a dynamic programming solution, but it exposes structural properties that will be exploited algorithmically.

3.8.4. Computational Implication

Even for a single decision window, the assignment space has cardinality $|\mathcal{K}|^N$, and budget constraints introduce knapsack-like coupling across VMs and, for some budget models, across time. As a result, the problem is combinatorial and computationally intractable at scale for exact methods. This observation motivates the complexity analysis in Section 4 and the proposed dual-variable-based solution strategy in Section 5, which leverages the per-VM structure while enforcing budget feasibility through a global shadow price.

4. Complexity Analysis

This section establishes the computational intractability of the *BC-VMR* problem formulated in Section 3.8. Since VM types are discrete and budget feasibility constraints couple decisions through realized costs, the resulting optimization problem is inherently combinatorial. The goal of this section is to position *BC-VMR* within the established combinatorial complexity landscape, rather than to claim a new hardness archetype. We prove NP-hardness for the scalarized family introduced in Equation (8), which immediately implies that the exact characterization of the Pareto frontier of the bi-objective problem in Equation (7) is intractable in the worst case.

4.1. NP-Hardness Analysis

To establish NP-hardness, we first consider the corresponding decision version of the scalarized problem in Equation (8).

Definition 1. Fix $\rho \geq 0$. Given an instance of *BC-VMR*(ρ) and a threshold Θ , decide whether there exists a feasible resizing plan $\mathbf{x}_{1:T} \in \mathcal{F}$ such that $F_{\rho}(\mathbf{x}_{1:T}) \leq \Theta$.

Lemma 1. *The decision problem in Definition 1 belongs to NP.*

Proof of Lemma 1. A certificate consists of a concrete resizing plan $\mathbf{x}_{1:T}$. Feasibility can be verified by evaluating constraints (3) and (4), where the cost function $\text{Cost}_t(\cdot)$ follows Section 3.2 and the budget function $g_t(\cdot)$ is instantiated by one of the models in Sections 3.7.1–3.7.4. The objective value is computed using Equations (1) and (2) and substituted into Equation (8). All required computations are polynomial in the number of VMs N , the number of windows T , and the size of the VM type set $|\mathcal{K}|$. $\square$

We next show that the BC-VMR decision problem is computationally intractable even under a highly simplified setting. In particular, we focus on the run-rate budget model, which imposes a per-window cost cap and represents the most basic form of budget constraint. By reducing from the classical 0–1 Knapsack decision problem, we demonstrate that this restricted case is already NP-complete.

Lemma 2. *Under the run-rate budget model (Section 3.7.1), the decision problem in Definition 1 is NP-complete. Hence, BC-VMR(ρ) is NP-hard even when $T = 1$, the stability objective (2) is inactive, and only a single resource dimension contributes to Equation (1).*

Proof of Lemma 2. We reduce from the 0–1 Knapsack decision problem. Consider a knapsack instance with items $i = 1, \dots, N$, weights p_i , values v_i , capacity C , and target value V . We construct a BC-VMR instance as follows.

We set $T = 1$ and activate a single resource dimension (Section 3.5.3), so that the scalarized objective in Equation (8) reduces to minimizing Equation (1). For the reduction, we fix the trade-off parameters by setting $\lambda = 1$ and $w_r = 1$ for the single active resource dimension. Moreover, we fix the overload amplification factor to a constant $\omega_{i,t,r} = \bar{\omega} \geq 1$ for all i, t . This choice only scales all values uniformly and does not affect NP-hardness. Define the spacing constant $M := 1 + \sum_{i=1}^N v_i + \max_i v_i$, $q_i = \mu_i = i \cdot M$. If the overload amplification factor in Section 3.5.1 satisfies $\omega(\cdot) \geq 1$, we fix all VMs to share a constant value $\bar{\omega}$; this uniformly scales all v_i and does not affect NP-hardness.

For each item i , we introduce two VM types:

$$\text{cap}(b_i) = i \cdot M, \quad \text{cost}(b_i) = c_0 + p_i, \quad \text{cap}(a_i) = iM - v_i, \quad \text{cost}(a_i) = c_0,$$

where $c_0 > 0$ is a constant baseline cost consistent with Section 3.2. The single-window budget is set to $B' := Nc_0 + C$.

Assigning type b_i yields zero waste and zero tail shortfall, whereas assigning type a_i yields a tail shortfall of exactly v_i with zero waste. Any mismatch assignment using a type intended for a different index $j \neq i$ incurs a penalty exceeding $\sum_k v_k$. Specifically, if $j < i$, the allocated capacity is below q_i , causing a tail shortfall of at least $(i - j)M \geq M$; if $j > i$, the allocated capacity exceeds μ_i , resulting in waste of at least $M - \max v \geq 1 + \sum_k v_k$. Therefore, for all thresholds $\Theta \leq \sum_i v_i$, we may restrict attention to assignments selecting only $\{a_i, b_i\}$ for each VM.

Let $z_i \in \{0, 1\}$ indicate whether b_i is selected. The run-rate budget constraint reduces to

$$\sum_i \text{cost}(a_i) + \sum_i p_i z_i \leq B' \iff \sum_i p_i z_i \leq C,$$

and the objective satisfies

$$F_\rho = \sum_i v_i (1 - z_i) \leq \sum_i v_i - V \iff \sum_i v_i z_i \geq V.$$

Thus, the knapsack instance admits a feasible solution if and only if the constructed BC-VMR(ρ) instance has a feasible plan with $F_\rho \leq \sum_i v_i - V$. The reduction is polynomial, and together with Lemma 1, the decision problem is NP-complete. $\square$

Corollary 1. *Under the absolute cumulative budget model (Section 3.7.2), BC-VMR(ρ) is NP-hard.*

Proof of Corollary 1. For $T = 1$, the cumulative constraint reduces to a single-window budget cap identical to the run-rate model. Lemma 2 applies directly. $\square$

Corollary 2. *Under the incremental budget model (Section 3.7.3), BC-VMR(ρ) is NP-hard.*

Proof of Corollary 2. Consider $T = 2$ and fix $\mathbf{x}_1$ such that $\text{Cost}_1(\mathbf{x}_1)$ is constant. The incremental constraint at $t = 2$ becomes a single-window cost cap up to an additive constant, reducing to the run-rate case. Lemma 2 applies. $\square$

Corollary 3. *Under the rolling budget model (Section 3.7.4), BC-VMR(ρ) is NP-hard.*

Proof of Corollary 3. Setting the rolling horizon to $W = 1$ reduces the rolling constraint to a per-window budget cap identical to the run-rate model. Lemma 2 applies. $\square$

Theorem 1. *For each budget model in Sections 3.7.1–3.7.4, computing a globally optimal solution of BC-VMR(ρ) is NP-hard. Consequently, exact computation of the Pareto frontier of BC-VMR (Section 3.8.1) is intractable in the worst case unless $P = NP$.*

Proof of Theorem 1. Lemma 2 establishes NP-hardness for the run-rate model. Corollaries 1–3 show that the remaining budget models each contain a parameter regime that reduces to the run-rate case. Therefore, all four instantiations yield NP-hard global optimization. $\square$

4.2. Implications and Discussion

The NP-hardness result holds even under extreme simplifications, including a single decision window, the absence of resizing-stability costs, and a single active resource dimension. The full multi-window BC-VMR problem with cross-window budget coupling is therefore at least as hard, motivating scalable approximation methods such as the dual-variable-based approach developed in Section 5.

Beyond the NP-hardness result, it is useful to characterize structural infeasibility more explicitly. Under the base run-rate model without additional dwell-time restrictions, let $k_{\min} \in \mathcal{K}$ denote the lowest-cost VM type in the pool. Then, window t is structurally infeasible whenever

$$N \cdot \text{cost}(k_{\min}) > B_{\text{rate_pw}}(t),$$

because even assigning every VM to the cheapest available type still exceeds the active budget cap. Equivalently, a necessary condition for window-level feasibility is

$$B_{\text{rate_pw}}(t) \geq N \cdot \text{cost}(k_{\min}).$$

This closed-form threshold explains the feasibility transition observed in Section 6: as the budget multiplier α increases, the feasible region expands until the run-rate budget becomes large enough to accommodate the lowest-cost assignment; beyond that point, feasible candidate decisions become available and violation rates collapse rapidly.

5. Proposed Method

This section presents the proposed Budget-aware Dual (BD) solver for the budget-constrained VM resizing problem. The goal is to obtain practically deployable solutions for BC-VMR(ρ) under hard budget feasibility, while preserving the core trade-off between sizing quality and resizing stability.

5.1. Problem Recap and Algorithmic Challenges

We consider the scalarized family BC-VMR(ρ) defined in Equation (8) over the feasible set $\mathcal{F}$ specified by constraints (3) and (4). The objective F_ρ admits the stage-wise representation in Equation (9), where the window-level terms depend on the current assignment $\mathbf{x}_t$ and, through the change indicator, on the previous assignment $\mathbf{x}_{t-1}$. Moreover, the waste and overload components within each window decompose across VMs and resource dimensions, revealing a per-VM separable structure inside f_1 .

Despite this structure, Section 4 establishes that computing a globally optimal solution of BC-VMR(ρ) is NP-hard under each budget model in Sections 3.7.1–3.7.4. Hence, exact methods, dynamic programming, or full Pareto-frontier enumeration are not viable at scale. Practical VM resizing controllers require (i) near-real-time decisions per window, (ii) scalability to large N , and (iii) strict budget awareness in the sense that budget policies remain feasibility requirements rather than additional objectives.

5.2. Key Idea: Dual-Based Budget Awareness Under Hard Constraints

The central difficulty is that budgets are imposed on realized spend, i.e., on the cost sequence $\{\text{Cost}_t(\mathbf{x}_t)\}_{t=1}^T$ through Equation (5) which is the generic feasibility constraint. Because VM types are discrete, it is possible that, in some windows, no assignment $\mathbf{x}_t \in \mathcal{K}^N$ satisfies the budget cap implied by g_t (e.g., due to a coarse type set or highly restrictive caps). In such regimes, budget violations are structurally unavoidable regardless of the algorithm. This observation motivates reporting the *Budget Violation Rate* in Section 6 as an operational metric, and it also motivates the need for a solver that reacts to budget pressure in a principled and stable way.

BD introduces a nonnegative dual variable ν that acts as a shadow price for the budget constraint. Importantly, ν is not used to redefine budgets as soft preferences. Instead, ν provides a global, quantitative signal of budget tightness that guides the selection toward lower-cost assignments, especially when the feasible set becomes empty in certain windows. Conceptually, when the budget is tight, ν increases and makes expensive VM types less attractive; when the budget is loose (or effectively inactive), ν decreases and ceases to throttle decisions.

From an operational perspective, budget policies are treated as hard feasibility requirements and define admissible operating regimes. Algorithmically, however, the discrete VM type set may render the feasible set empty in certain windows. In such cases, BD operates as an online primal–dual controller that reacts to budget pressure and minimizes the magnitude and frequency of unavoidable violations, rather than relaxing the constraint itself.

A key computational advantage follows: for a fixed value of ν at a given window, the resulting cost-aware decision can be made independently per VM, since the objective terms already admit a per-VM decomposition within each window. This yields a per-window runtime that scales approximately as $O(N|\mathcal{K}|)$, which is essential for the large-scale experiments in Section 6 and for practical deployment settings.

It is also informative to examine a simplified boundary case. When the run-rate (per-window) budget model is used and resizing stability is disabled, the decision in each window reduces to selecting one VM type for each VM under a single cost cap. This structure corresponds to a multiple-choice knapsack problem, where each VM forms a choice group and the controller selects one type per group subject to the budget limit. Such problems admit pseudo-polynomial dynamic programming algorithms and approximation schemes. However, once temporal coupling is introduced through cumulative, rolling, or incremental budget models, this stage-wise separability disappears and decisions become coupled across time. In these settings, scalable online control mechanisms such as the proposed BD solver become necessary.

5.3. Budget-Aware Reformulation of the Scalarized Problem

We start from BC-VMR(ρ) in Equation (8) under feasibility constraints (3) and (4). BD does not redefine the original optimization problem. Instead, it introduces a Lagrangian-style surrogate objective. For a given nonnegative dual sequence ν_t , we define the following surrogate cost function:

$$\min_{\mathbf{x}_{1:T}} F_\rho(\mathbf{x}_{1:T}) + \left| \sum_{t=1}^T \nu_t g_t(\mathbf{x}_{1:t}), \quad \nu_t \geq 0 \quad \forall t. \right. \quad (10)$$

For each budget model in Section 3.7, the window-wise residual can be expressed in an affine form with respect to the current-window cost,

$$g_t(\mathbf{x}_{1:t}) = \left| a_t \text{Cost}_t(\mathbf{x}_t) \right| + \left| c_t(\mathbf{x}_{1:t-1}), \right.$$

where $a_t \geq 0$ is a known coefficient (in all budget models considered here, $a_t = 1$) and c_t depends only on the realized history up to $t - 1$ and budget parameters (e.g., cumulative/rolling carry-over terms). Hence, in the primal step with fixed ν_t ,

$$\nu_t g_t = \left| \nu_t a_t \text{Cost}_t(\mathbf{x}_t) \right| + \left| \nu_t c_t, \right.$$

and the term $\nu_t c_t$ is constant with respect to $\mathbf{x}_t$ and can be dropped from the minimization. Finally, since $\text{Cost}_t(\mathbf{x}_t) = \sum_{i=1}^N \text{cost}(x_{i,t})$, the surrogate objective decomposes across VMs, yielding the per-VM selection rule in Equation (11).

This expression reuses the existing notions F_ρ , and $g_t(\cdot)$. For the rate-based, absolute cumulative, and rolling budget models, and for the incremental budget model when written in its equivalent affine form (Section 3.7.3), $g_t(\cdot)$ is affine in the current-window cost $\text{Cost}_t(\mathbf{x}_t)$ given the realized cost history. Therefore, the augmentation in Equation (10) introduces a linear cost pressure proportional to ν_t at each decision window. The dual variables ν_t are treated as exogenous parameters during the primal decision at each window. The BD algorithm alternates between minimizing this surrogate with respect to $\mathbf{x}_t$ for fixed ν_t , and updating ν_t via projected ascent based on the realized budget residual, as detailed in Algorithm 1.

Algorithm 1 BD (Budget-aware Dual) Solver

Input: VM type set $\mathcal{K}$, decision horizon T , previous assignment x_0 (or x_1 given), workload statistics $(\mu_{i,t,r}, q_{i,t,r}, \sigma_{i,t,r}, H_{i,t,r})$ as defined in Section 3.3, and a budget feasibility function $g_t(\cdot)$ from Section 3.7

1. Initialize the dual variable $v_1 \leftarrow 0$
2. For each decision window $t = 1, 2, \dots, T$:
 - 2.1 For each VM $i = 1, 2, \dots, N$:
 - 2.1.1 Evaluate the local augmented objective induced by the surrogate formulation in Equation (10)
 - 2.1.2 Select the VM type $x_{i,t}$ according to the per-VM decision rule defined in Equation (11), using the most recent workload statistics
 - 2.2 Compute the realized platform cost $\text{Cost}_t(\mathbf{x}_t)$ according to Section 3.2
 - 2.3 Update the cost history $\{\text{Cost}_1(\mathbf{x}_1), \text{Cost}_2(\mathbf{x}_2), \dots, \text{Cost}_t(\mathbf{x}_t)\}$
 - 2.4 Evaluate the budget residual $g_t(\mathbf{x}_{1:t})$
 - 2.5 Update the dual variable by projected ascent

$$v_{t+1} \leftarrow \min \left(\left[v_t + \eta \tilde{g}_t \right]^+, v_{\max} \right).$$

$v_{\max} > 0$ is a fixed upper bound used to prevent numerical explosion of the dual variable.

Output: sequential resizing decisions $\{\mathbf{x}_t\}_{t=1}^T$

Under the per-window run-rate budget model defined in Section 3.7.1, the budget residual becomes stage-wise separable and depends only on the current-window assignment:

$$g_t^{\text{rate-pw}}(\mathbf{x}_t) = \text{Cost}_t(\mathbf{x}_t) - B_{\text{rate-pw}}(t)$$

In this case, the primal decision performed by BD can be interpreted as minimizing the per-window Lagrangian

$$L_t(\mathbf{x}_t, v) = \left(\text{Waste}_t(\mathbf{x}_t) + \lambda \text{Over}_t(\mathbf{x}_t) + \rho \sum_{i=1}^N \mathbf{I}\{x_{i,t} \neq x_{i,t-1}\} \right) + v g_t^{\text{rate-pw}}(\mathbf{x}_t), v \geq 0$$

where v represents the shadow price associated with the hard budget constraint. The corresponding dual function is defined as

$$d_t(v) = \min_{\mathbf{x}_t \in \mathcal{K}^N} L_t(\mathbf{x}_t, v)$$

which is concave in v , since it is the pointwise infimum of affine functions of v . For any minimizer $\mathbf{x}_t(v)$, a valid dual subgradient is given by the realized budget residual $g_t^{\text{rate-pw}}(\mathbf{x}_t)$. Consequently, the update used in Algorithm 1,

$$v_{t+1} = \min \left(\max \left(0, v_t + \eta \tilde{g}_t \right), v_{\max} \right),$$

can be interpreted as a projected dual subgradient-ascent step applied to the run-rate Lagrangian dual problem. Equivalently, this update admits the interpretation of a virtual-queue update, where the normalized residual $\tilde{g}_t$ is defined in Section 5.4.

The practical consequence is that, for a fixed v_t , the window-level decision reduces to a per-VM type selection rule. Using the per-resource overload and waste quantities from Section 3.5, VM i at window t selects

$$x_{i,t} \in \underset{k \in \mathcal{K}}{\text{argmin}} \left[\sum_{r \in \mathcal{R}} w_r(\text{waste}_{i,t,r}(k) + \lambda \text{over}_{i,t,r}(k)) + |\rho \mathbf{I}\{k \neq x_{i,t-1}\}| + |v_t \text{cost}(k)| \right], \quad (11)$$

where $\text{waste}_{i,t,r}(k)$ and $\text{over}_{i,t,r}(k)$ denote the quantities defined in Section 3.5, evaluated by fixing $x_{i,t} = k$ while keeping all other components of $\mathbf{x}_t$ unchanged in the per-VM selection step; $\text{cost}(k)$ follows Section 3.2. This rule makes explicit how v_t converts budget pressure into a simple additive bias toward cheaper instance types while preserving the existing $f_1 - f_2$ trade-off encoded by the first two terms.

5.4. The BD Algorithm

We now describe the online BD procedure that alternates between (i) selecting $\mathbf{x}_t$ using the cost-aware per-VM rule and (ii) updating the dual variable based on the realized budget residual. The method operates in a sliding-window fashion consistent with the interpretation in Section 3.8.3.

The dual variable is updated based on the realized budget residual, which is normalized by a characteristic budget scale in practice to ensure dimensional consistency and numerical stability. The normalized residual is defined as

$$\tilde{g}_t := \frac{g_t}{\bar{B}},$$

where $\bar{B}$ denotes a characteristic budget scale associated with the active budget model. Here, the realized budget residual is computed with respect to the post-gate deployed configuration, ensuring that the dual update reflects execution-level feasibility rather than the raw candidate output. An additional upper projection $v_{\max}$ is applied to prevent numerical explosion and to ensure stable cost-aware behavior over long horizons and under cumulative budget models, with the concrete normalization used in the experiments described in Section 6. The resulting primal step has complexity $O(N|\mathcal{K}|)$ per window, since each VM evaluates a finite number of candidate types, enabling BD to maintain practical runtimes even at large N , as examined in Section 6.

5.5. Theoretical Properties of the Dual Variable

We summarize two key properties of the dual variable that motivate the experimental validation in Section 6.2. These properties are guided by the standard shadow-price interpretation of Lagrangian multipliers and serve as design targets for the dual-based solver. Because BC-VMR is a discrete NP-hard combinatorial problem, we do not claim the statements below as KKT guarantees for the original program; rather, they describe the expected shadow-price behavior of the dual-control signal induced by the surrogate primal–dual update in Algorithm 1 and are validated empirically in Section 6.2.

Proposition 1. *Consider two budget settings that differ only by a relaxation of the budget limits in $g_t(\cdot)$ (i.e., the constraint becomes weakly easier to satisfy for all t). Then the corresponding steady-state dual variable level v^* is expected not to increase under this relaxation. Equivalently, tighter budgets typically induce larger shadow prices, while looser budgets typically induce smaller shadow prices. Throughout this proposition, the budget residual underlying v is interpreted as the realized residual after the common compliance gate, so that the dual variable consistently reflects deployment-level feasibility rather than candidate-level outcomes.*

Proof of Proposition 1. Under a relaxed budget, any resizing plan feasible under the tighter budget remains feasible, and the budget residual $g_t(\cdot)$ becomes weakly smaller for the same cost sequence. In the dual interpretation, v measures the marginal benefit of relaxing the budget constraint; when the constraint is relaxed exogenously, this marginal value is not expected to increase. $\square$

Proposition 2. *If the budget constraint is inactive (i.e., the realized costs satisfy $g_t(\cdot) < 0$ over the relevant horizon), then the corresponding dual variable is expected to approach zero, $v^* \approx 0$; conversely, a strictly positive dual variable indicates that the budget constraint is effectively active.*

Proof of Proposition 2. When the budget constraint is inactive, additional relaxation provides no benefit. Hence the shadow price should be zero; otherwise, decreasing v would strictly improve the augmented objective without harming feasibility. $\square$

Section 6.2 empirically validates these two properties by examining how the measured dual variable responds to systematic budget tightening/relaxation and whether it converges to near-zero in well-funded regimes.

5.6. Minimal Primal–Dual Bounds in the Run-Rate Case

We next analyze the theoretical behavior of the BD update under the run-rate budget model. Recall that Algorithm 1 performs a projected subgradient ascent step on the dual variable v_t , where the subgradient corresponds to the normalized residual $\tilde{g}_t$ defined in Section 3.7.1. This residual measures the instantaneous deviation between the realized spending rate and the permitted run-rate budget. The analysis below characterizes two fundamental properties of this update rule. First, the dual sequence v_t achieves a standard subgradient regret bound relative to the best fixed dual value in hindsight. Second, the resulting primal decisions exhibit vanishing average budget violation over time. These guarantees provide a minimal theoretical justification that the BD update acts as a stable budget-regulation mechanism in the run-rate regime. Formally, the following theorem establishes regret and violation bounds for the dual update.

Theorem 2. *Under the continuous relaxation of the run-rate budget model (Section 3.7.1), and assuming exact minimization of the relaxed per-window primal subproblem, suppose that the normalized residual satisfies $|\tilde{g}_t| \leq G$ for all t . Then the dual iterates v_t generated by Algorithm 1 satisfy*

$$\frac{1}{T} [\max_{0 \leq v \leq v_{\max}} \Sigma d_t(v) - \Sigma d_t(v_t)] \leq \frac{v_{\max}^2}{2\eta T} + \frac{\eta G^2}{2}.$$

Moreover, the normalized time-average violation magnitude satisfies

$$\frac{1}{T} \Sigma [\tilde{g}_t]_+ = O\left(\frac{1}{\sqrt{T}}\right).$$

Proof of Theorem 2. Under the continuous run-rate relaxation, the update in Algorithm 1 corresponds to projected subgradient ascent applied to the concave dual function $d_t(v)$. Since $d_t(v)$ is concave and the normalized residual $\tilde{g}_t$ is bounded by G , the classical regret bound for projected subgradient methods over the compact domain $[0, v_{\max}]$ applies, which establishes the claimed inequality. Choosing $\eta = \frac{v_{\max}}{G\sqrt{T}}$ yields an $O\left(\frac{1}{\sqrt{T}}\right)$ dual regret rate. The same primal–dual argument implies that the time-average violation magnitude also decays at rate $O\left(\frac{1}{\sqrt{T}}\right)$. Under the continuous relaxation, the time-average objective approaches the relaxed optimum at the same rate. $\square$

Theorem 2 applies specifically to the run-rate budget model under a continuous relaxation, where the budget constraint is stage-wise separable and the dual update corresponds to a standard projected subgradient method. For temporally coupled budget models such as the absolute cumulative, rolling, and incremental budgets, feasibility depends on the realized cost history and the stage-wise separability used in the analysis no longer holds. Consequently, we do not claim analogous regret or violation guarantees for those mod-

els. Their behavior is therefore evaluated empirically in Section 6 rather than established through formal guarantees.

6. Results and Discussion

This section evaluates the proposed budget-constrained VM resizing framework and the BD solver. The experiments are designed in four stages. In the first stage, we validate the fundamental theoretical claims of the framework by empirically examining whether the dual variable v_t exhibits the monotonicity and complementary-slackness-like behavior expected from the shadow-price interpretation in Section 5. This step ensures that the Lagrangian-based formulation operates as intended when embedded within a dynamic workload environment. In the second stage, we investigate the scalability and computational complexity of BD by measuring its execution time as the number of managed VMs increases from tens to tens of thousands. This analysis addresses a critical requirement for real-world applicability, as hyperscale cloud platforms must perform resizing decisions within strict latency budgets and cannot rely on solvers whose runtime grows superlinearly with the system size. The third stage compares BD against several representative baseline algorithms—Static, Greedy, PRG, and NSGA-II—under multiple budget regimes. By evaluating cost saving, multi-objective utility, and budget feasibility, we assess how effectively BD balances the core FinOps trade-offs between economic efficiency, performance preservation, and budget compliance. Finally, in the fourth stage, we examine the robustness of BD when the underlying workload characteristics change. To this end, we construct representative workload scenarios with varying temporal variability. The primary quantitative evaluation is conducted on a Typical workload, while additional scenario-based analyses examine transient, sustained, and oscillatory demand dynamics. This stage evaluates whether BD maintains consistent behavior across diverse operating conditions or whether its performance is sensitive to the structure of workload fluctuations.

6.1. Experimental Environment

All experiments are conducted on a VM resizing simulator that implements the optimization problem defined in Section 3. The simulator operates in a sliding-window fashion. For each time window, we compute for every VM the mean, 95th percentile, standard deviation, and entropy of CPU and memory demand. To ensure dimensional consistency with the capacity terms $\text{cap}_r(\cdot)$ in Section 3, our workload generator produces absolute resource-demand samples $d_{i,t,r}(l)$ (measured in vCPU for $r = \text{cpu}$ and GiB for $r = \text{mem}$).

Given an assigned type $x_{i,t}$, the corresponding utilization ratio is computed only for reporting and monitoring purposes as $u_{i,t,r}(l) = \min\left(\frac{d_{i,t,r}(l)}{\text{cap}_r(x_{i,t})}, 1\right)$, $r \in \{\text{cpu}, \text{mem}\}$. Importantly, all window-level statistics $(\mu_{i,t,r}, q_{i,t,r}, \sigma_{i,t,r}, H_{i,t,r})$ used by the model are computed from the absolute demand samples $d_{i,t,r}(l)$ and therefore do not depend on the chosen VM type $x_{i,t}$; $u_{i,t,r}(l)$ is not used in any objective, constraint, or gate evaluation. Based on these statistics, it selects one specification from a finite pool of VM types. Each specification is characterized by its vCPU count, memory capacity, and an hourly price normalized from the price table of a public cloud provider. In our implementation, the VM-type pool spans from 1 vCPU/1 GiB to 16 vCPU/32 GiB and is ordered by increasing cost. The pool follows a coarse-grained progression at the low end and includes a small number of mid-tier diagonal types that scale CPU and memory asymmetrically, preventing the feasible set from degenerating to a single specification under transient workload shifts. The implementation is written in Python 3.x and runs on the same server across all experiments (Intel Xeon Silver-class CPU with 64 GB of RAM), so that execution times of different algorithms are directly comparable. At the beginning of window t , we compute work-

load statistics from the immediately preceding window $t - 1$ and choose the assignment $\mathbf{x}_t$ accordingly.

In the experimental evaluation, the workload-dependent amplification factor $\omega(\sigma, H)$ is instantiated using a lightweight bounded additive form. For each VM i , window t , and resource r , we set

$$\omega_{i,t,r} = 1 + \beta_1 \cdot \min\left\{1, \left|\frac{\sigma_{i,t,r}}{\text{cap}_r^{\max}}\right|\right\} + \beta_2 \cdot H_{i,t,r}$$

where $H_{i,t,r} \in [0, 1]$ is the normalized entropy defined in Section 3. This instantiation is monotone in σ and H , bounded for numerical stability, and depends only on observed workload statistics (not on the chosen VM type). The additive structure is used intentionally rather than as an arbitrary convenience. It preserves a direct interpretation of β_1 and β_2 as separate amplification strengths for volatility and irregularity, while avoiding the instability that can arise from multiplicative interactions when both signals are simultaneously large. The coefficient β_1 controls sensitivity to workload volatility (intra-window variability), while β_2 controls sensitivity to workload irregularity captured by entropy. In all experiments, we fix $\beta_1 = 0.3$ and $\beta_2 = 0.4$ to give slightly more weight to irregularity than to short-term variance, while keeping the amplification bounded and easy to interpret. This choice ensures monotonic risk amplification and numerical stability, consistent with the monotonicity and boundedness requirements described in Section 3.5.1.

6.1.1. Behavior-Driven Workload Generation

To evaluate the resizing framework under realistic and diverse operating conditions, we employ a behavior-based workload generator that produces detailed CPU and memory time series for every virtual machine. Rather than assuming a simple average utilization level, the generator explicitly constructs temporal patterns that reflect how cloud services vary over time. Four behavior types are implemented, capturing a wide range of demand dynamics observed in practice.

The first type, referred to as the static pattern, models service whose resource usage remains nearly constant. For these virtual machines, each sample in the time series is drawn from a narrow Gaussian distribution around a fixed mean, resulting in windows that exhibit low variance and stable p95 values. This pattern serves as a baseline for understanding how the algorithms behave when demand is predictable.

The second type corresponds to ramp-up and ramp-down behaviors. Here, the workload gradually increases or decreases by interpolating between a sampled start level and a sampled end level over a configurable number of intervals. Once the transition is completed, the workload either stabilizes at the new level or fluctuates mildly around it. This form of controlled non-stationarity introduces windows in which mean demand and p95 evolve continuously, thereby testing whether the solver reacts smoothly to gradual demand shifts.

The third type is the idle-burst behavior, designed to capture applications that spend long periods at very low utilization but occasionally experience intense demand spikes. The generator creates alternating idle and burst segments, drawing idle samples from a narrow distribution near zero and burst samples from a distribution with both higher mean and larger variance. Because the duration and magnitude of bursts are randomized, the resulting time series contains sudden peaks that increase entropy and widen the gap between mean and p95. This pattern stresses the resizing logic by forcing it to balance long idle periods against rare but critical bursts.

The fourth type, periodic-burst, introduces regular cyclic spikes. The generator constructs repeating cycles, each consisting of an idle phase followed by a burst phase. Since the cycle length and burst duration follow consistent values across repetitions, the resulting

workload exhibits predictable oscillations. This allows us to examine whether the solver can exploit periodic structure to anticipate future demand rather than reacting solely to instantaneous fluctuations.

Demand traces are generated in absolute units (vCPU and GiB) by scaling each behavior's utilization ratios by the global caps of the type of pool (16 vCPU and 32 GiB). Static traces follow a Gaussian model with low mean and small variance, while ramp-up/down traces linearly interpolate between sampled start and end levels and then continue with mild noise. Burst-type traces alternate between idle segments and high-demand segments: idle-burst uses irregular burst intervals and durations, whereas periodic-burst repeats a fixed period with a short burst phase. Across all behavior types, CPU and memory traces are generated independently using the same template and then scaled to each VM capacity units. For monitoring, utilization ratios are capped at 1, while the underlying demand samples remain unclipped. The simulator computes per-window statistics (mean, standard deviation, 95th percentile, and entropy) from these demand traces, and these window-level statistics are fed into the objective terms and feasibility checks, influencing overload-risk evaluation and headroom requirements. By assigning heterogeneous behavior types across VMs and randomizing burst timing, ramp duration, amplitude, and noise levels, the aggregate workload spans both smooth and highly volatile regimes, exercising resizing decisions under steady demand, gradual transitions, abrupt spikes, and periodic fluctuations.

Unless otherwise stated, the workload generator samples its parameters from fixed ranges that are shared across all experiments. Burst intervals are drawn uniformly from 5 to 60 min, burst durations from 5 to 30 min, and burst amplitudes from 0.6 to 1.2 of the VM capacity. Ramp-up and ramp-down traces use ramp lengths between 30 and 120 min, while Gaussian noise is added with a standard deviation of at most 10% of the mean demand level. For scenario construction, the workload mixture is fixed per scenario: the Steady workload consists entirely of static traces, the Typical workload uses a mixture of 50% static, 30% ramp (20% ramp-up and 10% ramp-down), and 20% idle-burst traces, and the Bursty workload uses 30% ramp, 50% idle-burst, and 20% periodic-burst traces.

6.1.2. Experimental Budget Constraints

Budget constraints are modeled primarily using two of the four budget models in Section 3.7: the run-rate budget $B_{\text{rate_pw}}$ and the absolute cumulative budget $B_{\text{abs_cum}}$. The strength of the budget is controlled by a scalar multiplier α applied to a calibrated base budget B_0 :

$$B(\alpha) = \alpha \cdot B_0.$$

The base budget B_0 is calibrated once using the Static policy on a reference workload realization and is then held fixed across all evaluation runs and random seeds. This design reflects practical FinOps settings in which budgets are specified in advance based on a reference calibration run and are not adjusted to each realized workload trace. Consequently, even the Static policy may violate the nominal budget level under $\alpha = 1.0$ on some realizations.

For the run-rate budget, the base level is calibrated from the peak (maximum) per-window cost observed under the Static policy in the calibration run.

$$B_0^{\text{rate}} := \max_t \text{Cost}_t(\mathbf{x}^{\text{static}}), \quad B_{\text{rate_pw}} = \alpha \cdot B_0^{\text{rate}}.$$

In the absolute cumulative model, the entire budget allowance is available from the beginning of the horizon and is consumed cumulatively over time, rather than being released according to a predefined trajectory.

$$B_0^{\text{cum}} := \sum_{t=1}^T \text{Cost}_t(\mathbf{x}^{\text{static}}), \sum_{t=1}^T \text{Cost}_t(\mathbf{x}_t) \leq \alpha \cdot B_0^{\text{cum}}.$$

For the incremental budget, we use a time-invariant per-window increment cap over the horizon $B_{\text{inc_cum}}(t) := \alpha \cdot B_0^{\text{inc}}$ for all $t \geq 2$. Because the Static policy keeps configurations fixed and would yield a degenerate zero increment under direct calibration, we set $B_0^{\text{inc}} := B_0^{\text{rate}}$ (the calibrated peak per-window spend) as a common reference scale. With this choice, the multiplier α directly controls the allowed relative growth per window under the incremental budget model.

For the dual update in the BD solver, we use a normalized budget residual to obtain a dimensionless quantity. Specifically, the raw residual g_t , defined in Section 3.7, is scaled by a characteristic budget level as

$$\tilde{g}_t := \frac{g_t}{\bar{B}}, \bar{B} \in \{B_0^{\text{rate}}, B_0^{\text{cum}}\}.$$

Here, $\bar{B}$ denotes the calibrated base budget corresponding to the active budget model (run-rate or absolute cumulative). This normalization ensures dimensional consistency of the dual variable and improves numerical stability across different budget models and budget magnitudes. For the incremental budget, we reuse $\bar{B} = B_0^{\text{rate}} = B_0^{\text{inc}}$ for normalization.

In all experiments, we use a fixed step size $\eta = 1.0$ for the dual update. Because the budget residual is normalized, a single step size was sufficient to obtain stable behavior across all budget models.

We sweep $\alpha \in \{0.1, 0.2, \dots, 1.0\}$ to study the effect of budget strength. Unless stated otherwise, all reported results are obtained from this full α sweep. Because the budget is fixed across realizations, this level may still be insufficient for some workload traces, and constraint violations can persist. The run-rate and absolute cumulative models are used as canonical extremes in the main experiments; we additionally include the incremental cumulative model in Section 6.3 to contrast growth-limiting policies with absolute spending caps, while the rolling model is discussed in Section 6.2.

6.1.3. Baseline Algorithms and Proposed Solver

Since BC-VMR(ρ) is NP-hard even under highly simplified settings (Section 4), we focus on scalable heuristic and dual-based solvers rather than exact optimization. Accordingly, the baselines include lightweight heuristics and a meta-heuristic (NSGA-II) that approximates the Pareto front but is not intended for real-time deployment. Under this setting, we evaluate a total of five representative algorithms that reflect different design trade-offs between optimality, scalability, and operational practicality. The *Static* policy keeps the initial specification for each VM throughout the entire horizon and serves as a QoS upper bound that ignores cost and budget. *Greedy* is a simple downsizing-only heuristic that, in each window, selects a lower-cost VM specification when doing so improves the sizing-quality objective; it does not explicitly incorporate the budget constraint during candidate generation. *BD* is the proposed dual-variable solver. *Proportionate Ratio Greedy* (PRG) selects VM specifications based on a utility-to-cost ratio, and likewise generates candidates without directly enforcing the budget constraint. At each window, Greedy and PRG generate candidates independently for each VM based only on the current window statistics. Neither baseline enforces the budget constraint during candidate generation;

budget feasibility is applied uniformly afterward through the common compliance gate. Ties are broken deterministically by selecting the lowest-cost specification. *NSGA-II* is a multi-objective evolutionary algorithm and acts as an optimization-quality baseline rather than a practical online policy. At each decision window t , *NSGA-II* searches over the current assignment vector $\mathbf{x}_t$ (one gene per VM representing the selected type), evaluates the objectives using the window-level statistics from $t - 1$, and computes the stability term based on whether each VM changes relative to its previous assignment $x_{i,t-1}$; the best feasible individual (if any) is used as the candidate decision before applying the common compliance gate. For scalability experiments we only consider BD, Greedy, and *NSGA-II* because they represent, respectively, the proposed solver, a very lightweight heuristic, and a heavy but powerful meta-heuristic.

All algorithms operate online and use only statistics computed from the most recent completed window. For evaluation, we apply the common compliance gate described above to the candidate decision produced by each method. Concretely, when the candidate violates the budget while a feasible assignment exists, the gate deterministically repairs the candidate by (i) keeping the current specification when it is already budget-feasible, (ii) otherwise replacing the highest-cost VM assignments with the globally cheapest types in the VM-type pool, and (iii) if needed, iteratively downsizing the remaining highest-cost VMs until the total cost satisfies the constraint. For *NSGA-II*, the candidate is chosen by feasibility-first selection within the nondominated set (the best feasible individual if one exists); the selected candidate is then also passed through the same compliance gate to ensure that feasibility is interpreted identically across all methods. Unless otherwise stated, *NSGA-II* uses the fixed setting in Table 3 ($pop = 50$, $gen = 50$, $p_c = 0.7$, $p_m = 0.3$), corresponding to 2500 fitness evaluations per decision window, with no early stopping or wall-clock truncation.

6.1.4. Evaluation Protocol

To avoid ambiguity between a candidate decision and the final deployable action, we evaluate every method using the same two-step protocol in each decision window. First, the algorithm produces a candidate assignment based on the most recent workload statistics. Second, the candidate is passed through a common budget-compliance gate that checks the active temporal budget policy. If the candidate violates the budget but a feasible assignment exists within the discrete VM-type pool, the gate deterministically repairs the decision by progressively reducing cost while preserving the algorithm's intent as much as possible (see the deterministic procedure in Section 6.1.3). If no feasible assignment exists for that window under the given budget and VM-type pool (i.e., the active budget is violated even by the lowest-cost assignment in the VM-type pool), the decision is marked as structurally infeasible and the repaired action becomes the lowest-cost assignment available; any remaining overspend is then unavoidable by definition. Unless stated otherwise, all reported metrics are computed from the final post-gate decisions, since these correspond to what would actually be executed in an operational controller. This gate is applied immediately after each algorithm produces its candidate decision for window t and before any cost accumulation, budget-residual evaluation, or metric computation.

6.1.5. Performance Metrics

We report three primary metrics throughout the experiments.

Cost Saving is defined as the percentage reduction in the time-weighted total cost relative to the Static policy. This metric directly quantifies the economic benefit achieved under budget constraints.

Budget Violation Rate (post-gate) is defined as the fraction of decision windows in which the final decision after the common compliance gate still violates the active budget constraint. This metric reflects deployment-level feasibility. For diagnostic purposes, we also report the *Candidate Violation Rate* (pre-gate), defined as the fraction of windows in which the candidate decision violates the budget when evaluated before the gate (i.e., $g_t(\cdot) > 0$ on the candidate). Finally, the *Structural Infeasibility Rate* is the fraction of windows in which even the minimum-cost assignment in the discrete VM-type pool violates the cap, in which case any post-gate overspend is unavoidable by definition. Unless stated otherwise, cost saving and stability metrics are computed from post-gate decisions.

Operational Stability is measured using the change rate, defined as the fraction of virtual machines whose assigned type differs from the previous window after applying the compliance gate. This metric captures operational churn associated with resizing actions and reflects migration overhead and potential service disruption. Change rates are computed with respect to the final post-gate deployed configurations, ensuring consistency with execution-level feasibility and the evaluation protocol in Section 6.1.4.

For each configuration, we run 10 independent random seeds and report mean $\pm$ standard deviation. Unless otherwise stated, all algorithms are evaluated using the same seed set. Table 3 summarizes the experimental configuration and hyperparameters used across Sections 6.2–6.4. Table 4 lists the discrete VM-type pool used in our experiments, including vCPU count, memory size (GB), and per-hour price (normalized units). Prices in Table 4 are normalized by dividing raw hourly prices by the cheapest type price and scaling to integer units for readability; all budgets are calibrated in the same normalized units. While we fix this monitoring granularity for controlled comparison, the formulation and simulator directly support finer sampling (larger L) without changing any algorithmic component.

Table 4. Discrete VM-type pool $\mathcal{K}$ used in the simulator (15 types) with per-hour price p_k (normalized units).

vCPU c_k	vMemory (GB) m_k	Price/h p_k
1	1	60
2	2	100
2	3	130
2	4	160
4	8	300
4	16	420
6	16	520
8	16	600
10	20	720
10	24	820
12	24	880
12	28	1000
14	28	1080
14	32	1160
16	32	1200

6.2. Theoretical Validation

6.2.1. Monotonicity and Complementary Slackness of the Dual Variable

This section verifies whether the theoretical properties of the dual variable ν derived in Section 5 are indeed observed in the simulator. Specifically, we assess whether the empirically measured steady-state dual level ν^* decreases monotonically as the budget is relaxed, and whether ν^* approaches zero when the budget constraint becomes inactive. Unless stated otherwise, ν^* is computed as the mean of ν_t after discarding the initial 20% of windows as a burn-in period. All workload, cost, and specification settings are fixed; only the budget multiplier α is varied from 0.0 to 1.0 in increments of 0.1. In each run, the BD algorithm updates the per-window dual variable ν_t , and we record the mean of ν_t over the remaining windows as ν^* . At the same time, we measure the pre-gate constraint violation rate as the fraction of decision windows in which the corresponding budget constraint is violated when evaluated on the candidate decision, i.e., $g_t(\cdot) > 0$.

For the run-rate budget, Figure 1 reports the empirically measured steady-state dual level ν^* together with the candidate (pre-gate) budget violation rate as functions of the budget multiplier α . Under tight budgets, ν^* remains extremely large, and the candidate violation rate is high, indicating strong budget pressure. As α increases, ν^* decreases monotonically, consistent with the shadow-price monotonicity intuition ($\frac{\partial \nu}{\partial B} \leq 0$).

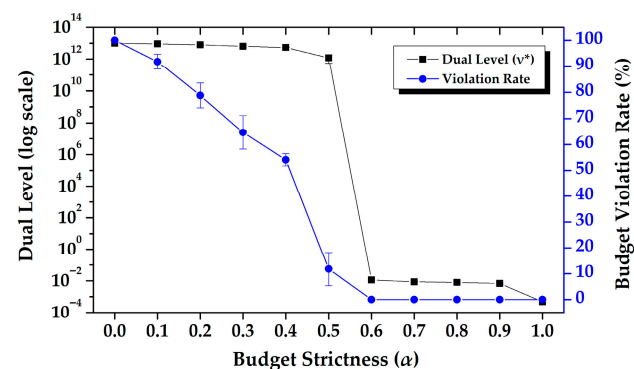


Figure 1. Dual level and candidate feasibility under the run-rate (per-window) budget. Steady-state dual level ν^* (shadow price of budget pressure) and candidate (pre-gate) budget violation rate as functions of the budget multiplier α under the run-rate budget model $B_{\text{rate_pw}}$. α controls budget strictness (smaller α = tighter budget). Dual axis uses a log scale.

Around $\alpha \approx 0.6$, the candidate violation rate rapidly drops to near zero, indicating that most candidate decisions become feasible. This feasibility knee is consistent with the structural interpretation discussed in Section 4: once the budget becomes large enough to accommodate the lowest-cost assignment in the VM-type pool, feasible candidate decisions become available and violation rates collapse rapidly. In lockstep, ν^* decreases by several orders of magnitude and converges toward zero, which is empirically consistent with the complementary-slackness intuition

$$\nu \cdot g(x) = 0,$$

suggesting that the dual variable tends to become negligible when the corresponding constraint becomes inactive. In this low- α (tight-budget) regime, a subset of windows can be structurally infeasible under the discrete VM-type pool (i.e., even the minimum-cost assignment violates the cap), in which case post-gate overspend is unavoidable by definition; accordingly, we use the candidate (pre-gate) violation curve here as a diagnostic of primal-dual behavior, while deployment-level feasibility is assessed using post-gate outcomes in Section 6.3. Practically, Figure 1 shows that BD does not impose unnecessary throttling in

well-funded regimes: once the budget becomes sufficiently relaxed, the algorithm drives the penalty toward zero and stops interfering with the primal decisions.

Figure 2 presents the same validation for the absolute cumulative budget model. We again plot the steady-state dual level ν^* and the candidate (pre-gate) budget violation rate as functions of α . In contrast to the run-rate budget, the violation rate decreases more gradually as α increases, reflecting the history-coupled nature of cumulative feasibility under a fixed ex-ante calibrated cap. Consistent with Proposition 1, ν^* decreases as the budget is relaxed, remaining large while the cumulative constraint is frequently active. Once α reaches the point where the cumulative constraint becomes effectively slack (near $\alpha = 1.0$ in this sweep), the violation rate approaches zero and ν^* collapses toward zero by several orders of magnitude. This indicates that the monotonicity behavior of the dual variable is not only observed in the rate-based model but also under structurally different temporal budget formulations.

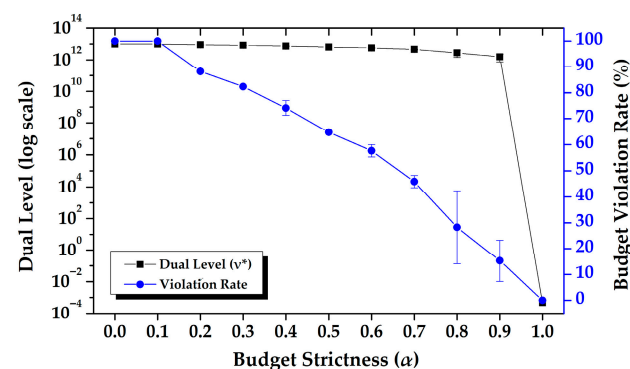


Figure 2. Dual level and candidate feasibility under the absolute cumulative budget. Steady-state dual level ν^* and candidate (pre-gate) budget violation rate versus α under the absolute cumulative budget model B_{abs_cum} , illustrating history-coupled feasibility and the corresponding decay of ν^* as the constraint becomes slack.

Applying the same α -sweep procedure to additional budget variants yields qualitatively consistent behavior. In particular, the incremental cumulative model reinforces the same monotonic trends while adding a distinct growth-limiting perspective to the comparison. For the main empirical trade-off analysis, however, we focus on the per-window run-rate, absolute cumulative, and incremental cumulative budgets, because these three families provide the clearest representative contrasts among local spending caps, long-horizon cumulative limits, and upsizing-driven growth constraints. We treat the rolling-budget variant only as a supplementary diagnostic rather than as a primary comparison axis, which keeps the main comparison focused while still supporting the broader applicability of the framework. Across the additional variants, relaxing the budget drives the dual variable toward zero and reduces the violation rate, whereas tightening the budget increases both the dual variable and budget pressure, consistent with the monotonic trends observed under the run-rate and absolute cumulative models. Overall, these results reinforce the same qualitative picture captured by the main comparison families and support the theoretical foundations of the proposed framework. This diagnostic focuses on candidate pre-gate violations to relate dual-variable behavior to budget pressure; deployment-level feasibility is reflected by the post-gate outcome discussed in Section 6.1.5.

6.2.2. Scalability Analysis

This section examines the computational scalability of the resizing algorithms as the fleet size increases. We vary the number of VMs N from 50 to 10,000 and measure

the runtime required by BD, Greedy, and NSGA-II to compute resizing decisions under identical hardware and software conditions.

Figure 3 summarizes runtime scalability by plotting the per-decision execution time against fleet size $N \in \{50, \dots, 10,000\}$ on log–log axes. For each N , we report both the average and the 95th-percentile (p95) wall-clock runtime across repeated runs, so that the figure not only captures typical decision latency but also tail-latency behavior relevant to real controllers. The magnitude gap is already visible at small scales: at $N = 50$, NSGA-II requires on the order of $\sim 10^3$ ms per decision (avg; p95 slightly higher), whereas BD remains around a few milliseconds and Greedy stays below 1 ms. As N grows, this separation widens rather than collapses. At $N = 1000$, NSGA-II rises to roughly $\sim 10^4$ ms, while BD is still only tens of milliseconds and Greedy remains within the single-digit milliseconds range. At the largest scale $N = 10,000$, NSGA-II reaches approximately $\sim 10^5$ ms, whereas BD stays around $\sim 10^2$ – 10^3 ms and Greedy around $\sim 10^1$ – 10^2 ms, yielding a persistent high two-digit to low three-digit advantage of BD over NSGA-II even at hyperscale.

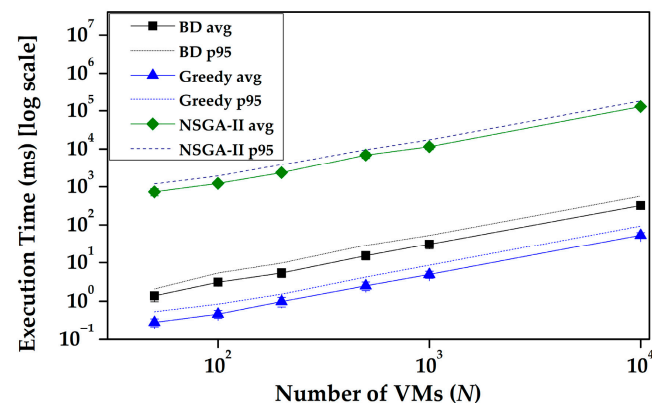


Figure 3. Execution Time versus Number of Virtual Machines. Per-decision wall-clock runtime versus fleet size N (log–log axes) for BD, Greedy, and NSGA-II. Lines report average and p95 decision latency across repeated runs under identical hardware/software settings.

Two trends stand out from these concrete values. First, NSGA-II exhibits a steep growth curve as N increases, reflecting the inherent computational burden of evolutionary multi-objective search over an N -dimensional discrete assignment vector. Second, BD scales much more gently and remains close to linear growth, consistent with the per-window complexity $O(N|K|)$ induced by the per-VM selection rule in Equation (11) and the lightweight dual update. The Greedy heuristic remains the fastest among the three, but its slope grows more noticeably with N than BD, while still remaining far below the evolutionary baseline. Importantly, the gap between average and p95 runtimes is small for BD across the sweep, indicating stable execution time and limited tail-latency amplification even at large scales. Overall, Figure 3 demonstrates that BD maintains practical decision latency at fleet sizes where NSGA-II becomes computationally prohibitive, underscoring BD’s suitability for large-scale VM resizing scenarios while preserving a substantial runtime advantage across the tested range. Accordingly, Figure 3 should be interpreted as a fleet-size scalability result under a fixed VM-type pool ($|\mathcal{K}| = 15$), a fixed decision horizon ($T = 18$), and fixed NSGA-II settings ($pop = 50$, $gen = 50$, i.e., 2500 fitness evaluations per decision window). It is not intended as a budget-equalized comparison across solver paradigms. We also do not model explicit migration latency, control-plane overhead, or memory footprint; the current results should therefore be read as controller-level optimization evidence rather than an end-to-end deployment study.

Beyond fleet size, VM resizing decisions in practice are also influenced by the size of the available VM-type pool $|\mathcal{K}|$ and the temporal planning horizon T . These two dimensions influence both the computational scalability of the controller and the efficiency of feasible assignments within the discrete configuration space. We therefore further examine how BD behaves as $|\mathcal{K}|$ and T vary, and how the granularity of the VM-type pool affects the efficiency of feasible configurations.

Table 5 shows that the per-window runtime of BD grows predictably with $|\mathcal{K}|$. At the baseline horizon $T = 18$, increasing the pool size from $|\mathcal{K}| = 6$ to 10 and 15 raises the mean runtime from 14.65 ± 0.73 to 19.92 ± 0.68 and 26.71 ± 0.80 ms/window. By contrast, at the baseline pool size $|\mathcal{K}| = 15$, increasing the horizon from $T = 9$ to $T = 36$ changes the per-window runtime only modestly, from 26.51 ± 1.36 to 28.42 ± 1.59 ms/window, whereas the total horizon runtime grows from 238.55 ± 12.26 to 1022.99 ± 57.35 ms. This behavior is consistent with the $O(N|\mathcal{K}|)$ per-window structure implied by Equation (11) and with the online sliding-window implementation of BD.

Table 5. Computational scalability of BD with respect to VM-type pool size and decision horizon.

Configuration	$ \mathcal{K} $	T	Runtime (ms/Window)	Runtime (ms/Horizon)
Baseline	15	18	26.71 ± 0.80	480.71 ± 14.46
Reduced VM-type pool	6	18	14.65 ± 0.73	263.67 ± 13.22
Moderate VM-type pool	10	18	19.92 ± 0.68	358.51 ± 12.27
Short Horizon	15	9	26.51 ± 1.36	238.55 ± 12.26
Long Horizon	15	36	28.42 ± 1.59	1022.99 ± 57.35
Small pool + long horizon	6	36	14.90 ± 0.65	536.53 ± 23.40

Table 6 examines the structural impact of VM-pool granularity. Using identical workload snapshots, we compute the cheapest feasible assignment available within each candidate VM-type pool. Relative to the full pool ($|\mathcal{K}| = 15$), reducing the pool to $|\mathcal{K}| = 10$ and $|\mathcal{K}| = 6$ increases the oracle feasible-cost gap to $8.08 \pm 0.63\%$ and $11.17 \pm 1.06\%$, respectively, while the excess normalized slack gap rises to $3.70 \pm 0.35\%$ and $4.83 \pm 0.29\%$. The infeasible snapshot rate remains $28.92 \pm 3.65\%$ across all tested pools because each sub-pool retains the maximum-capacity VM type. Consequently, the effect of pool granularity appears not in binary feasibility but in the efficiency of feasible assignments.

Table 6. Structural impact of VM-type pool granularity.

$ \mathcal{K} $	Oracle Feasible Cost/Snapshot	Cost Gap vs. Full Pool (%)	Excess Slack Gap (%)	Infeasible Snapshot Rate (%)
6	835.79 ± 21.56	11.17 ± 1.06	4.83 ± 0.29	28.92 ± 3.65
10	823.67 ± 21.08	8.08 ± 0.63	3.70 ± 0.35	28.92 ± 3.65
15	782.15 ± 24.00	0.00 ± 0.00	0.00 ± 0.00	28.92 ± 3.65

Taken together, these results indicate that the resizing framework maintains predictable controller-level scalability with respect to $|\mathcal{K}|$ and T , while the discrete VM-type pool introduces measurable discretization loss when the pool becomes coarse.

6.2.3. Primal–Dual Lower Bound and Relaxation-to-Integer Gap

To connect the relaxation-based bounds in Section 5.6 to the original discrete decisions of BD, we quantify a primal–dual lower bound in the run-rate model and report the

associated relaxation-to-integer gap. Let $d_t(\nu)$ denote the run-rate dual function introduced in Section 5.3, and let

$$\ell_t(\mathbf{x}_t) := \text{Waste}_t(\mathbf{x}_t) + \lambda \text{Over}_t(\mathbf{x}_t) + \rho \sum_{i=1}^N \mathbb{I}\{x_{i,t} \neq x_{i,t-1}\}$$

denote the corresponding per-window primal objective in Equation (9). We approximate the dual lower bound $d_t^* = \max_{0 \leq \nu \leq \nu_{\max}} d_t(\nu)$ via a simple 1D grid search over ν (log-spaced), which is inexpensive because the inner minimization reuses the same per-VM scan over $\mathcal{K}$ as Equation (11). We then report the relative primal–dual gap for BD (candidate decisions) as

$$\text{Gap}_t(\%) = 100 \times \frac{\ell_t(\mathbf{x}_t) - d_t^*}{\max[1, |d_t^*|]},$$

and summarize the mean/median of Gap_t over windows and seeds. We report this metric only for windows that are not structurally infeasible under the run-rate budget. Across these windows, 95.29% of Gap_t values are exactly 0% (p99 = 23.18%), indicating that the dual lower bound is typically tight in the feasible run-rate regime.

6.3. Budget-Aware Performance Trade-Offs

This section evaluates budget-aware performance trade-offs on a representative Typical workload using three temporal hard-budget models: (a) absolute cumulative $B_{\text{abs_cum}}$, (b) per-window run-rate $B_{\text{rate_pw}}$, and (c) incremental cumulative $B_{\text{inc_cum}}$. The Typical workload combines static demand with ramp transitions and intermittent bursts, reflecting realistic mixed dynamics. We compare four online algorithms—Static, Greedy, PRG, and the proposed BD—under ten random seeds. We set $\rho = 10$ to emphasize resizing stability by penalizing frequent configuration changes. We sweep the common budget multiplier $\alpha \in \{0.1, 0.2, \dots, 1.0\}$. Unless stated otherwise, cost and stability metrics in this section are computed from the final post-gate decisions produced by the common compliance gate (Section 6.1.4). For budget feasibility, we report the Candidate Violation Rate (pre-gate) to quantify how strongly each method internalizes the active budget policy.

We focus the main comparison on the per-window run-rate budget, the absolute cumulative budget, and the incremental cumulative budget because these three families provide the clearest representative contrasts among temporal hard-budget semantics. The per-window run-rate budget isolates strictly local spending control and is also the analytically most tractable case due to window-wise separability. The absolute cumulative budget represents fully history-coupled budget control over the horizon, while the incremental cumulative budget captures a distinct growth-limiting regime that constrains upsizing-driven cost increases rather than total accumulated spend. Taken together, these three models span the principal trade-off patterns of interest in our study—stage-wise caps, long-horizon cumulative limits, and growth-constrained resizing—while keeping the empirical comparison focused and interpretable. We therefore treat the rolling-budget variant only as a supplementary diagnostic rather than as a primary comparison family in this section.

6.3.1. Cost Efficiency

Figure 4 reports cost saving relative to Static as the budget multiplier $\alpha \in 0.1, 0.2, \dots, 1.0$ varies, revealing markedly different behaviors across temporal hard-budget semantics. Under the per-window run-rate model (Figure 4b), BD shows a pronounced plateau followed by a clear knee: savings remain essentially unchanged at about $\sim 49.3\%$ over $\alpha = 0.1$ – 0.4 , then gradually decline as the budget relaxes—46.4% at $\alpha = 0.5$, 40.2% at $\alpha = 0.6$, 31.6% at $\alpha = 0.7$, and 26.5% at $\alpha = 0.8$ —before dropping further to 15.5% at $\alpha = 0.9$.

and 2.9% at $\alpha = 1.0$. PRG attains even larger savings in the very tight regime (e.g., 88.2% at $\alpha = 0.1$, 77.2% at $\alpha = 0.2$, and 56.9% at $\alpha = 0.4$), but steadily converges toward BD as the budget loosens (e.g., 39.9% at $\alpha = 0.6$, 27.5% at $\alpha = 0.8$) and becomes similarly small at $\alpha = 1.0$ (1.6%). In contrast, Greedy remains nearly constant at 36.0% across all α , indicating weak responsiveness to budget tightening or relaxation when savings are measured relative to Static.

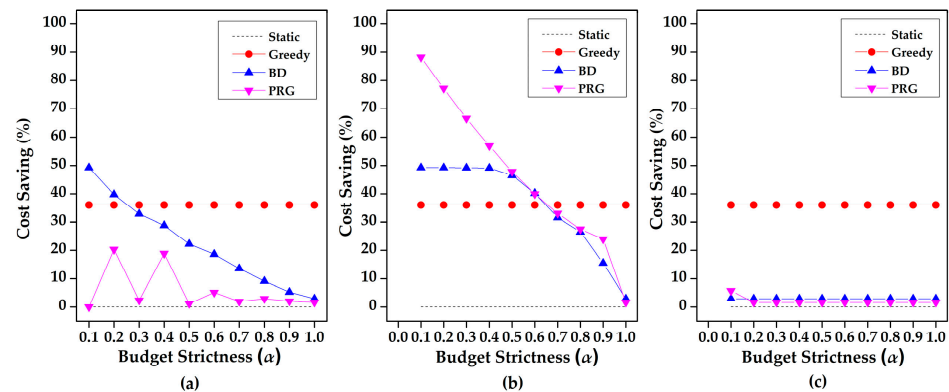


Figure 4. Cost saving vs. budget strength α (post-gate). Cost saving (%) relative to Static as a function of the budget multiplier α under (a) absolute cumulative $B_{\text{abs_cum}}$, (b) run-rate per-window $B_{\text{rate_pw}}$, and (c) incremental cumulative $B_{\text{inc_cum}}$; metrics are computed from post-gate (deployable) decisions (mean $\pm$ std over 10 seeds).

Under the absolute cumulative model (Figure 4a), savings are more strongly shaped by history coupling. BD decreases smoothly from 49.3% at $\alpha = 0.1$ to 39.7% ($\alpha = 0.2$), 28.8% ($\alpha = 0.4$), 18.6% ($\alpha = 0.6$), 9.1% ($\alpha = 0.8$), and 2.9% ($\alpha = 1.0$). PRG is notably less consistent: it achieves moderate savings at some tight-budget points (e.g., 20.3% at $\alpha = 0.2$ and 18.8% at $\alpha = 0.4$) but collapses to single-digit savings for looser budgets (e.g., 5.1% at $\alpha = 0.6$) and becomes very small by $\alpha = 1.0$ (1.6%). Greedy again appears almost budget-insensitive (about 36.0% across α), but this apparent advantage must be interpreted jointly with feasibility and stability outcomes, because cumulative constraints can reward aggressive early downsizing that reshapes the remaining feasible cost trajectory.

Finally, under the incremental model (Figure 4c), BD and PRG show consistently small cost savings across the sweep: BD stays at about 2.9% for all α , while PRG is 5.7% at $\alpha = 0.1$ and approximately 1.6% for $\alpha \geq 0.2$. Greedy remains nearly constant at 36.0%. This pattern is consistent with the incremental budget semantics in our simulator, which charge only positive (scale-up) cost increases and ignore savings from downsizing when computing feasibility; as a result, varying α has limited leverage on the realized cost savings for stability-aware policies in this setting ($\rho = 10$).

Overall, Figure 4 suggests that the proposed BD most effectively converts budget flexibility into economic benefit under the run-rate model, while the cumulative and incremental families highlight why savings alone are insufficient and must be evaluated together with feasibility and operational stability (Sections 6.3.2–6.3.4).

6.3.2. Budget Feasibility

In this section, we evaluate budget feasibility under temporal hard-budget policies using a diagnostic view that isolates algorithmic budget awareness from the shared deployment mechanism. The candidate violation rate is defined as the fraction of decision windows in which the candidate decision violates the active budget constraint before applying the common compliance gate. This metric quantifies how strongly each method internalizes the active budget policy (i.e., how much it relies on the gate to become deploy-

able). Deployment-level feasibility is captured by the post-gate violation rate (Section 6.1.5): under our deterministic common gate, any avoidable violation is repaired whenever a feasible assignment exists in the discrete VM-type pool; therefore, post-gate violations occur only in structurally infeasible windows. Accordingly, we focus on the pre-gate candidate violation rate as a regime diagnostic.

Figure 5b shows that feasibility under the per-window run-rate model $B_{\text{rate_pw}}$ exhibits a sharp threshold behavior consistent with its stage-wise separable nature. BD transitions rapidly from frequent violations under very tight budgets to a feasible regime with essentially zero violations: its candidate violation rate is around the low-to-mid 90% range at $\alpha = 0.1$, drops to roughly the mid-50% range by $\alpha = 0.4$, falls to around the low teens near $\alpha = 0.5$, and reaches 0.0% once $\alpha \gtrsim 0.6$. A clear feasibility knee is therefore observed around $\alpha \approx 0.6$, beyond which violations remain at 0.0%. In contrast, Greedy remains infeasible deeper into the sweep: it stays above roughly 50% through $\alpha = 0.5$ and still shows substantial violations around $\alpha = 0.6$, only reaching 0.0% around $\alpha \approx 0.7$. PRG maintains 0.0% violations across all values of α , while Static violates the run-rate budget severely throughout most of the range (remaining above 50% even near $\alpha = 0.9$) and reaches 0.0% only at $\alpha = 1.0$. These results indicate that BD achieves strict feasibility in a practically relevant regime where non-budget-aware heuristics continue to incur frequent violations, effectively converting budget relaxation into operationally executable decisions rather than merely reducing cost.

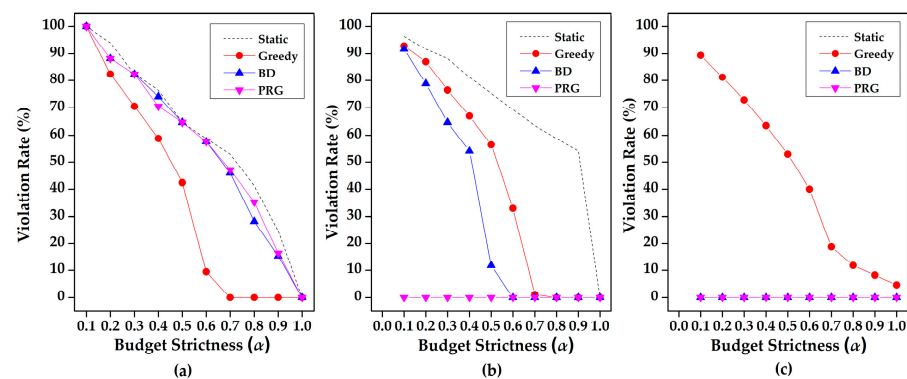


Figure 5. Candidate budget violation rate vs. budget strength α (pre-gate). Candidate (pre-gate) violation rate (%) versus α under (a) absolute cumulative $B_{\text{abs_cum}}$, (b) per-window run-rate $B_{\text{rate_pw}}$, and (c) incremental cumulative $B_{\text{inc_cum}}$ (mean $\pm$ std over 10 seeds). In (c), the Static violation rate is 0 for all α , so the dotted Static curve is not separately visible.

Figure 5a illustrates that feasibility under the absolute cumulative budget $B_{\text{abs_cum}}$ is substantially more challenging due to history coupling, whereby early spending decisions constrain the remaining feasible cost trajectory. Under this model, BD and PRG remain highly infeasible across tight-to-moderate regimes and only approach 0.0% violations at the fully nominal setting $\alpha = 1.0$, reflecting the intrinsic difficulty of satisfying a cumulative constraint when the trajectory is coupled over time. Greedy, by contrast, reduces violations much faster and reaches 0.0% earlier (around $\alpha \approx 0.7$). However, such feasibility improvements must be interpreted jointly with operational stability (Section 6.3.3), as cumulative budgets can favor strategies that reshape the cost trajectory via aggressive early downsizing and frequent reconfiguration.

Finally, Figure 5c shows that the incremental cumulative budget $B_{\text{inc_cum}}$, which primarily constrains upsizing increments (i.e., the total positive cost increases caused by scale-up actions within a window), yields a distinct feasibility pattern. Static is trivially feasible under these semantics, and both BD and PRG maintain 0.0% violations across the entire sweep. In contrast, Greedy violates the incremental constraint frequently under tight

budgets (near 90% around $\alpha = 0.1$) and improves only gradually as α increases, remaining nonzero even at $\alpha = 1.0$. This behavior indicates that even cost-efficient heuristics can trigger repeated upsizing spikes that violate growth constraints, whereas BD preserves strict feasibility without relying on aggressive reconfiguration.

Overall, Figure 5 confirms that the feasibility advantage of BD is most pronounced under the run-rate budget $B_{\text{rate_pw}}$. A clear feasibility knee appears around $\alpha \approx 0.6$, beyond which BD achieves 0.0% candidate violations (for $\alpha \geq 0.6$) while still maintaining strong cost savings. This transition aligns with the structural feasibility threshold discussed in Section 4, where the run-rate budget becomes sufficient to accommodate the lowest-cost assignment in the VM-type pool. Under cumulative budgets, feasibility becomes inherently history-dependent and must therefore be interpreted jointly with operational stability. In contrast, under incremental budgets feasibility is typically maintained by stability-aware policies, although non-budget-aware heuristics may still violate the growth constraint.

6.3.3. Operational Stability

In this section, we evaluate operational stability by analyzing the change rate, defined as the fraction of virtual machines whose configuration is modified within a decision window. While frequent reconfiguration can help reduce cost or enforce feasibility, excessive changes incur migration overhead, control-plane load, and potential service disruption. Therefore, a practically desirable method should maintain a low change rate while still achieving cost efficiency and budget feasibility. Our objective is to observe how change rates evolve as the budget is relaxed and how this evolution depends on the semantics of the budget model.

Figure 6b shows that under the per-window run-rate budget model $B_{\text{rate_pw}}$, BD maintains consistently low change rates across the entire sweep of α . Concretely, BD stays in the low teens under tight budgets (around 12% at $\alpha = 0.1$), then decreases steadily as the budget relaxes, reaching only a few percent by $\alpha \approx 0.6$ and approaching about 1% at $\alpha = 1.0$. In contrast, PRG exhibits strongly increasing churn as α grows: it rises from the low teens at $\alpha = 0.1$ to exceed 60% in the moderate-to-loose regime and further exceeds 70% for $\alpha \gtrsim 0.7$. Greedy remains nearly constant at roughly 52% regardless of α , indicating persistent aggressive reconfiguration. Static trivially shows a 0.0% change rate but, as shown in Section 6.3.2, fails to maintain feasibility under tight budgets. Together, these results demonstrate that BD uniquely combines feasibility (Figure 5b) with operational stability under a stage-wise separable budget, whereas PRG and Greedy rely on frequent reconfiguration to satisfy constraints or extract savings.

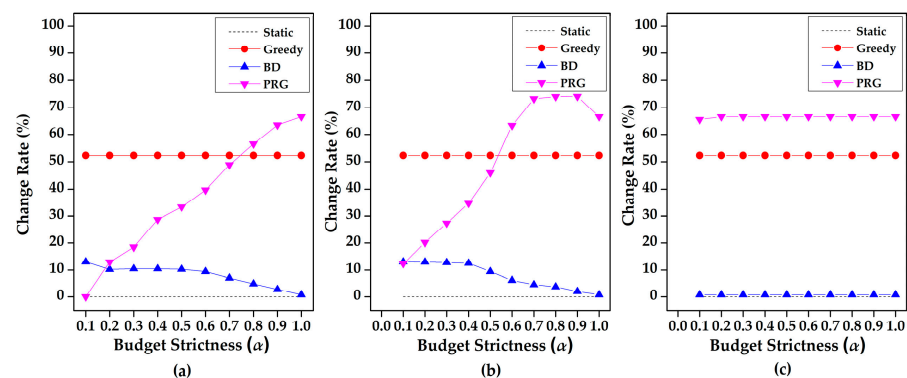


Figure 6. Change rate vs. budget strength α (post-gate). Post-gate change rate (%) versus α under (a) absolute cumulative $B_{\text{abs_cum}}$, (b) per-window run-rate $B_{\text{rate_pw}}$, and (c) incremental cumulative $B_{\text{inc_cum}}$ (mean $\pm$ std over 10 seeds).

Under the absolute cumulative budget model B_{abs_cum} (Figure 6a), history coupling amplifies instability for feasibility-oriented heuristics. BD again remains stable, with change rates around the low teens under tight budgets and decreasing toward near-zero as α approaches 1.0. By contrast, PRG shows a steady increase in churn as the budget relaxes, reaching the mid-60% range by $\alpha = 1.0$. Greedy again stays flat at about 52%, consistent with its budget-insensitive behavior. This pattern indicates that under cumulative constraints, PRG effectively trades stability for feasibility by repeatedly reshaping the cost trajectory, whereas BD preserves stability while gradually transitioning toward feasibility.

Figure 6c presents the incremental cumulative budget model B_{inc_cum} , where the budget primarily throttles cost increases. In this setting, BD maintains an almost negligible and nearly flat change rate (about 1%) across all values of α , indicating strong stability. PRG and Greedy, however, show persistently high change rates—around the mid-60% and low-50% ranges, respectively—despite the incremental constraint. This highlights that suppressing cost escalations alone does not prevent excessive reconfiguration for heuristics that lack explicit stability awareness.

Overall, Figure 6 demonstrates that BD consistently achieves low operational churn across all budget families, while PRG and Greedy rely on frequent reconfiguration to achieve feasibility or cost reduction. Combined with the feasibility results in Section 6.3.2 and the cost efficiency trends in Section 6.3.1, these findings confirm that BD attains a favorable balance between cost savings, budget compliance, and operational stability—an essential property for practical, large-scale VM management.

6.3.4. Trade-Off Decomposition

Cost efficiency, budget feasibility, and operational stability do not always improve simultaneously. Relying on a single metric can therefore obscure meaningful differences between algorithms. Figure 7 presents a decomposed view of the multi-objective outcome at a representative operating point with $\alpha = 0.6$ and $\rho = 10$ under three temporal hard-budget models B_{abs_cum} , B_{rate_pw} , and B_{inc_cum} . The value $\alpha = 0.6$ corresponds to a mid-budget regime near the feasibility transition observed in the budget sweep, and $\rho = 10$ maintains the stability emphasis used throughout the main evaluation.

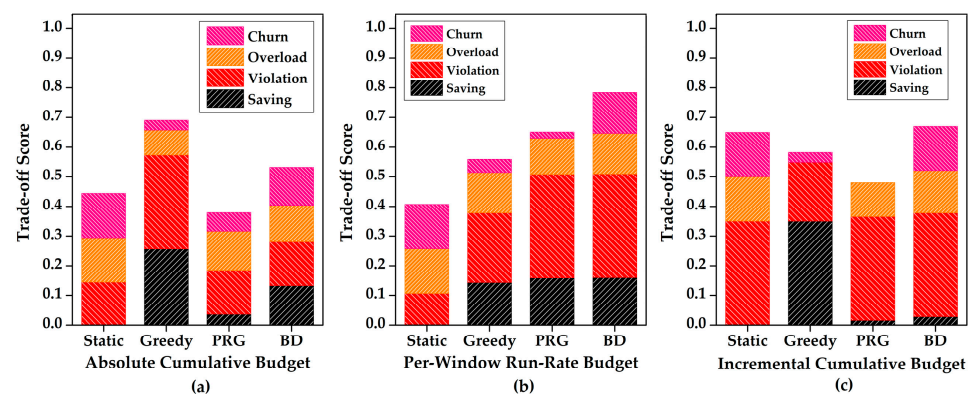


Figure 7. Normalized trade-off decomposition at $\alpha = 0.6$. Stacked components (cost saving, candidate violation, overload ratio, and churn) under (a) absolute cumulative B_{abs_cum} , (b) per-window run-rate B_{rate_pw} , and (c) incremental cumulative B_{inc_cum} ; values are min–max normalized within each budget model.

For each budget family, a composite score is constructed from four components derived from the same per-window statistics used in Sections 6.3.1–6.3.3. The components are cost saving relative to Static, candidate pre-gate violation rate, weighted overload ratio, and operational churn computed from change rate and migrations per virtual machine. Each

component is min–max normalized to the interval $[0, 1]$ within the same budget family across all algorithms and α values. For violation, overload, and churn, the normalized value is inverted so that larger values consistently represent better performance. The trade-off score is then defined as

$$S_{\text{trade_off}} = 0.35 S_{\text{saving}} + 0.35 S_{\text{violation}} + 0.15 S_{\text{overload}} + 0.15 S_{\text{churn}}.$$

Figure 7 stacks these weighted contributions, and the total bar height corresponds to $S_{\text{trade_off}}$. The weighting emphasizes economic benefit and budget compliance while preserving meaningful contributions from overload risk and operational stability.

Under $B_{\text{abs_cum}}$, Greedy attains the largest composite score of approximately 0.690, mainly due to strong saving and feasibility contributions. However, its churn contribution remains very small at approximately 0.032, reflecting highly reactive resizing behavior. BD achieves a lower total of approximately 0.532 but exhibits a more balanced distribution, with a substantially larger churn share of approximately 0.129 and a comparable overload contribution. This profile better reflects deployment-oriented stability requirements.

Under $B_{\text{rate_pw}}$, the separation is more pronounced. BD achieves the highest score at approximately 0.784 by combining the maximal feasibility contribution of approximately 0.35 with meaningful saving and substantial overload and churn shares. PRG also achieves the maximal feasibility term but shows a much smaller churn contribution of approximately 0.021, indicating frequent resizing under strict per-window caps.

Under $B_{\text{inc_cum}}$, feasibility differences are reduced because the incremental constraint limits only positive cost increases. Once scale-up actions are controlled, candidate violations largely disappear and the violation component saturates. In this regime, ranking is primarily determined by stability-related terms. BD achieves the highest total score of approximately 0.669 because it maintains stronger churn and overload contributions than the other non-static methods while still preserving non-zero saving.

The decomposition clarifies the practical advantage of BD. Across different budget semantics, BD avoids dominance by a single metric and maintains a consistent balance between economic efficiency, budget awareness, and operational stability at a deployment-relevant operating point.

6.3.5. Parameter Sensitivity of BD

BD introduces several hyperparameters in the dual update (η and ν_{max}) as well as trade-off weights in the objective (λ , ρ , and $(w_{\text{cpu}}, w_{\text{mem}})$). To assess whether the results in Section 6.3 rely on delicate tuning, we conduct a local one-factor-at-a-time sensitivity analysis at a representative operating point: the Typical workload under the run-rate (per-window) budget with $\alpha = 0.6$ and a representative fleet size $N = 100$. We vary one parameter at a time while keeping all others fixed to the reference setting used for this analysis ($\eta = 1.0$, $\nu_{\text{max}} = 10^9$, $\lambda = 0.5$, $\rho = 10$, and $(w_{\text{cpu}}, w_{\text{mem}}) = (0.8, 0.2)$). For each setting, we run 10 random seeds and report cost saving, candidate (pre-gate) violation rate, and post-gate change rate as defined in Section 6.1.5. Results are summarized in Table 7.

Overall, BD remains fully feasible at the candidate level across all tested settings at this operating point, yielding 0.0% candidate violations (mean $\pm$ std = 0.0 ± 0.0) in Table 7. In this feasible run-rate regime, the dual-update hyperparameters η and ν_{max} show no measurable effect within the tested ranges. Specifically, sweeping η from 0.2 to 2.0 and ν_{max} from 10^3 to 10^9 leaves both cost saving and change rate essentially unchanged, at $48.64 \pm 1.68\%$ and $21.11 \pm 1.23\%$, respectively. This indicates that BD does not require fine-grained tuning of the dual step size or the dual projection bound to maintain feasibility and stable behavior once the run-rate budget enters a feasible region.

Table 7. Local sensitivity of BD hyperparameters under the Typical workload and the run-rate (per-window) budget at $\alpha = 0.6$. Fleet size $N = 100$. Reported values are mean $\pm$ standard deviation over 10 random seeds. Cost saving and change rate are computed from post-gate decisions, while candidate violation rate is computed from pre-gate candidate decisions.

Parameter	Value	Cost Saving (%)	Candidate Violation (%)	Change Rate (%)
η	0.2	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	0.5	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	1.0	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	2.0	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
$\nu_{\max}$	1×10^3	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	1×10^5	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	1×10^7	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	1×10^9	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
λ	0.1	50.71 ± 1.62	0.0 ± 0.0	38.52 ± 2.20
	0.3	49.57 ± 1.61	0.0 ± 0.0	26.20 ± 1.55
	0.5	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	1.0	48.59 ± 1.54	0.0 ± 0.0	17.28 ± 0.68
ρ	5	49.98 ± 1.68	0.0 ± 0.0	18.97 ± 1.29
	10	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23
	20	47.82 ± 1.48	0.0 ± 0.0	22.36 ± 1.26
$(w_{\text{cpu}}, w_{\text{mem}})$	(0.2, 0.8)	49.73 ± 1.69	0.0 ± 0.0	18.93 ± 1.11
	(0.5, 0.5)	48.76 ± 1.58	0.0 ± 0.0	19.79 ± 1.30
	(0.8, 0.2)	48.64 ± 1.68	0.0 ± 0.0	21.11 ± 1.23

In contrast, the overload emphasis λ materially affects resizing aggressiveness and, indirectly, operational churn. Increasing λ from 0.1 to 1.0 reduces the change rate from $38.52 \pm 2.20\%$ to $17.28 \pm 0.68\%$, while cost saving changes only modestly from $50.71 \pm 1.62\%$ to $48.59 \pm 1.54\%$. This indicates that stronger tail-safety weighting can suppress reactive resizing without materially eroding economic benefit at this operating point.

Varying the stability weight ρ and the resource aggregation weights $(w_{\text{cpu}}, w_{\text{mem}})$ changes the quantitative values slightly but does not alter the qualitative conclusions. Across $\rho \in \{5, 10, 20\}$, cost saving remains within a narrow band ($49.98 \pm 1.68\%$ at $\rho = 5$ versus $47.82 \pm 1.48\%$ at $\rho = 20$), and change rate varies moderately ($18.97 \pm 1.29\%$ at $\rho = 5$ versus $22.36 \pm 1.26\%$ at $\rho = 20$). Similarly, shifting the aggregation weights between CPU- and memory-emphasized settings yields only modest changes (e.g., $49.73 \pm 1.69\%$ saving and $18.93 \pm 1.11\%$ change rate at $(w_{\text{cpu}}, w_{\text{mem}}) = (0.2, 0.8)$, versus $48.64 \pm 1.68\%$ saving and $21.11 \pm 1.23\%$ change rate at $(0.8, 0.2)$). Taken together, Table 7 confirms that the main conclusions of Section 6.3 are robust to moderate hyperparameter variations. To assess whether the conclusions depend on the specific workload-risk instantiation, we also verified in supplementary robustness checks that varying the entropy resolution B_{bin} and the amplification weights (β_1, β_2) around the default setting. Across these variants, the feasibility transition and the qualitative ranking of BD, Greedy, and PRG remained unchanged, indicating that the main conclusions do not hinge on a single choice of entropy discretization or amplification strength. In particular, the run-rate feasibility knee

remained near $\alpha \approx 0.6$, while BD's post-gate cost saving changed only modestly across the tested variants.

6.4. Scenario-Based Behavioral Analysis

While Section 6.3 evaluates budget-aware performance under temporal hard constraints, those results intertwine intrinsic resizing behavior with compliance-gate feasibility repair and budget-induced throttling. In order to examine the structural behavioral properties of each policy independent of budget enforcement, we temporarily disable the temporal budget constraint in this section. This isolation allows us to analyze the intrinsic dynamics of resizing decisions without the confounding effects of feasibility correction or dual-induced budget pressure. Specifically, we seek to isolate three aspects of algorithmic behavior: economic responsiveness to workload variation, stability regulation through the change-penalty parameter ρ , and temporal reaction patterns under distinct workload dynamics. By removing budget feasibility constraints, we observe how each method internally balances the sizing-quality objective f_1 and the stability objective f_2 across qualitatively different demand regimes.

Across Figures 8–10, we evaluate Static and Greedy baselines, BD under $\rho \in \{0.5, 2, 5, 10\}$, and an offline NSGA-II reference. The central objective of this section is to demonstrate that BD forms a policy family parameterized by ρ , where ρ acts as an interpretable control knob that continuously shifts the operating point between responsiveness and conservatism. Rather than being tuned to a specific workload, BD exhibits structurally consistent behavior across transient spikes, sustained shifts, and oscillatory regimes.

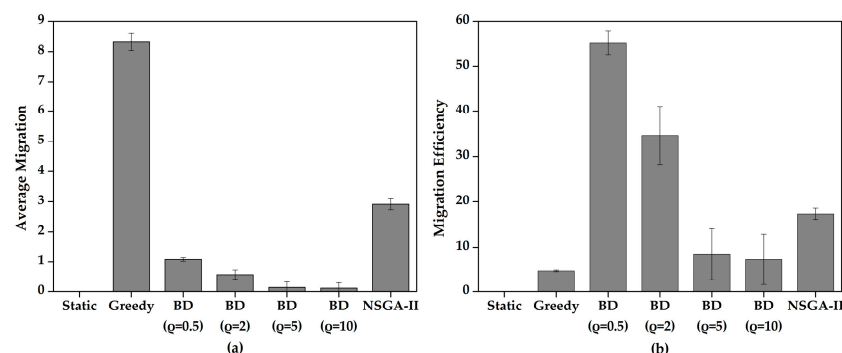


Figure 8. Transient spike workload: migration frequency and migration efficiency (budgets disabled). Behavioral response under a transient spike-dominated workload: (a) average number of migrations per VM and (b) migration efficiency (cost saving per migration) for Static, Greedy, BD with $\rho \in \{0.5, 2, 5, 10\}$, and NSGA-II. Budgets are disabled to isolate intrinsic stability–responsiveness behavior.

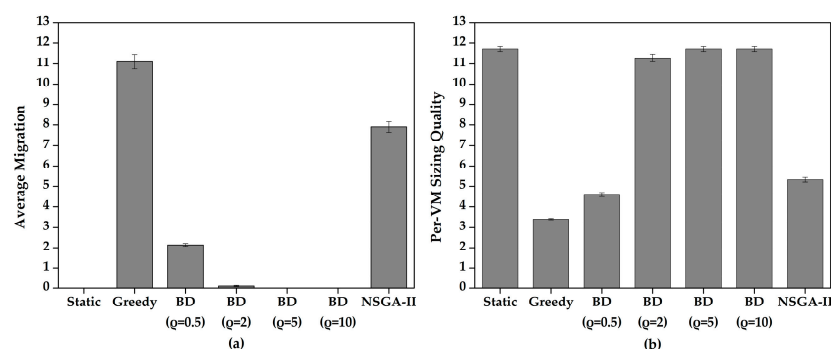


Figure 9. Sustained demand shift workload: adaptation vs. conservatism (budgets disabled). Behavioral response under a sustained ramp-down workload: (a) average number of migrations per VM and (b) per-VM f_1 (waste plus overload penalty) for Static, Greedy, BD with $\rho \in \{0.5, 2, 5, 10\}$, and NSGA-II. Lower f_1 indicates better sizing quality.

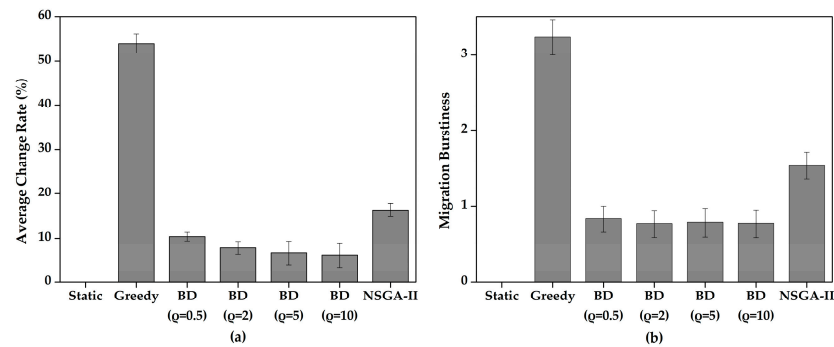


Figure 10. Oscillatory workload: churn and migration burstiness (budgets disabled). Behavioral response under an oscillatory workload: (a) average configuration change rate and (b) migration burstiness (standard deviation of per-VM migration counts) for Static, Greedy, BD with $\rho \in \{0.5, 2, 5, 10\}$, and NSGA-II.

6.4.1. Transient Spike-Dominated Workload

Figure 8 evaluates behavior under spike-dominated workloads, where short-lived bursts occur intermittently over a mostly low-utilization background. The primary failure mode in this regime is over-reactive resizing, in which the controller repeatedly upsizes and downsizes in response to ephemeral spikes, inducing excessive migration overhead.

Figure 8a reports the average number of migrations per virtual machine. Greedy exhibits heavy reconfiguration with 8.31 migrations per VM, indicating strong spike chasing. In contrast, BD at $\rho = 0.5$ reduces migration activity to 1.08 migrations per VM while simultaneously increasing cost saving from 38.71% (Greedy) to 59.48%. This indicates that BD can extract higher economic benefit with substantially fewer disruptive actions. As ρ increases, BD further suppresses migration activity (e.g., 0.57 at $\rho = 2$ and approximately 0.11–0.14 at $\rho \in \{5, 10\}$), illustrating a controllable shift toward conservative, low-churn behavior.

Figure 8b reports migration efficiency (cost saving per migration), defined as $\frac{\text{Cost Saving}(\%)}{\text{Average Migrations per VM}}$, which highlights the quality of resizing actions rather than their frequency. Greedy achieves only 4.66 saving-units per migration, whereas BD achieves 55.19 at $\rho = 0.5$ and remains high at $\rho = 2$ (34.63), demonstrating that BD converts each migration into substantially larger economic benefit. NSGA-II improves over Greedy in efficiency (17.35) but remains notably less migration-efficient than BD in this transient-spike regime. Overall, Figure 8 supports a key claim of the paper: BD avoids thrashing and achieves high economic benefit with minimal operational disruption, particularly when transient spikes dominate.

6.4.2. Sustained Shift Workload

Figure 9 evaluates behavior under sustained demand shifts with a gradual ramp-down pattern. In this regime, desirable behavior is controlled adaptation: the policy should downsize to remove persistent over-provisioning, but should not rely on aggressive, high-frequency reconfiguration.

Figure 9a reports average migrations per VM. Greedy achieves strong savings (48.30%) but at very high operational overhead (11.11 migrations per VM) and a high change rate (51.62%). BD at $\rho = 0.5$ preserves most of the savings (41.70%) while reducing migrations to 2.14 per VM and the change rate to 10.01%, demonstrating that stability-aware control can retain economic benefit without incurring persistent churn. As ρ increases, BD becomes progressively more conservative; at $\rho = 2$, the migration count drops to 0.10 per VM (change rate 0.47%), but savings collapse to 2.63%, indicating that excessive stability emphasis suppresses adaptation.

Figure 9b reports the per-VM value of f_1 (waste plus overload penalty), which reflects how well the policy aligns capacity to sustained demand. Greedy attains the lowest f_1 (3.41), but only by aggressive reconfiguration. BD at $\rho = 0.5$ achieves a competitive f_1 (4.59) while drastically reducing churn, whereas BD at $\rho = 2$ returns close to Static behavior ($f_1 = 11.29$ vs. 11.73 for Static). NSGA-II provides an offline reference with $f_1 = 5.32$ and 7.90 migrations per VM, showing that higher-quality solutions are possible but often at substantially higher operational cost than BD stability-favorable operating points. Overall, Figure 9 highlights a central message: ρ selects qualitatively different BD policies, and moderate ρ yields a practically deployable balance between adaptation and stability under sustained shifts.

6.4.3. Oscillatory Workload

Figure 10 evaluates behavior under oscillatory workloads that mix multiple variability sources. Such regimes often induce chattering, where frequent back-and-forth resizing produces operational instability with limited net benefit.

Figure 10a reports the average change rate. Greedy exhibits severe thrashing with a 53.95% change rate, while BD reduces the change rate by an order of magnitude for all tested ρ values (e.g., 10.39% at $\rho = 0.5$ and 7.80% at $\rho = 2$). NSGA-II also reduces churn relative to Greedy (16.25%) but remains notably less stable than BD in the balanced regime.

Figure 10b reports migration burstiness (the standard deviation of per-VM migration counts), capturing whether resizing actions are concentrated on a subset of VMs. Greedy shows strong concentration (3.23), indicating migration hotspots. BD substantially reduces burstiness (e.g., 0.83 at $\rho = 0.5$ and 0.77 at $\rho = 2$), showing that BD not only reduces overall churn but also avoids concentrating operational disruption. Importantly, BD achieves these stability gains while maintaining strong savings: 49.76% at $\rho = 0.5$ and 35.95% at $\rho = 2$, compared to 28.27% for Greedy. This confirms that stability-aware resizing can dominate reactive heuristics even in highly oscillatory environments.

6.5. Real-Trace Validation

6.5.1. Experimental Setup Using Real Trace

In this section, we conduct experiments using a real-world workload trace derived from the Google ClusterData (2011) dataset [45]. Specifically, we use the task_usage records to construct a VM-like fleet by treating each (job_id, task_index) pair as an individual workload instance. A contiguous 24-h slice of the trace is extracted and aggregated at 5-min intervals, resulting in 288 decision windows for a fleet of 50 VMs.

Because the original trace values represent normalized utilization ratios rather than absolute resource demand, we apply a simple calibration to align the trace with the VM capacity scale used in the simulator. Let cpu_{p95} and mem_{p95} denote the global 95th percentile of CPU and memory demand observed in the trace. These values are scaled so that the p95 demand corresponds to approximately 70% of the maximum VM capacity in the type pool (16 vCPU and 32 GB). This calibration preserves the temporal dynamics of the workload while ensuring that resizing decisions exercise a meaningful range of instance types. Using this calibrated trace, we evaluate the same set of policies used in the main experiments—Static, Greedy, PRG, and the proposed BD solver—under two representative temporal hard-budget models: the per-window run-rate budget and the absolute cumulative budget. The budget multiplier α is swept from 0.1 to 1.0 while the stability weight ρ is fixed at 2. All performance metrics follow the evaluation protocol defined in Section 6.1.5 and are computed from the final post-gate decisions produced by the common compliance gate.

6.5.2. Budget Trade-Offs Under Real Workloads

Figures 11–13 summarize the real-trace evaluation results obtained using the Google ClusterData workload.

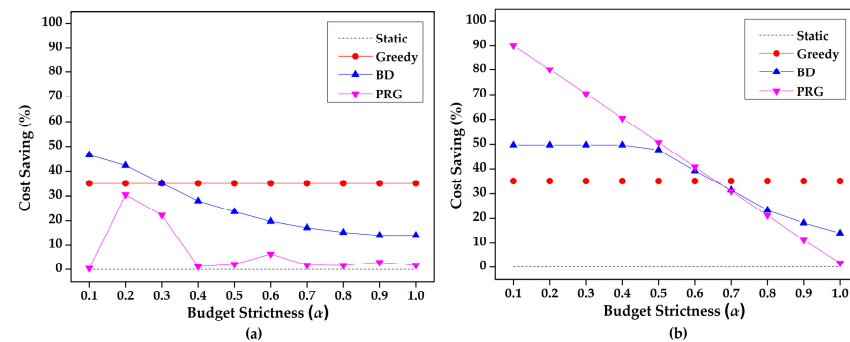


Figure 11. Real-trace cost saving vs. budget strictness (post-gate). Cost saving (%) relative to Static on the Google ClusterData (2011) [45] trace versus budget multiplier α under two temporal hard-budget models: (a) absolute cumulative B_{abs_cum} and (b) per-window run-rate B_{rate_pw} . All metrics are computed from post-gate (deployment-level) decisions produced by the common compliance gate; $\rho = 2$.

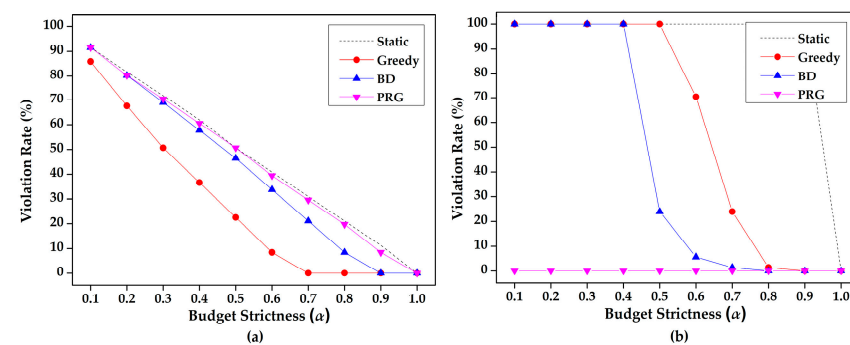


Figure 12. Real-trace budget feasibility vs. budget strictness (post-gate violation rate). Post-gate (deployment-level) budget violation rate (%) versus α on the Google ClusterData trace under (a) absolute cumulative B_{abs_cum} and (b) per-window run-rate B_{rate_pw} . Post-gate violations represent windows that remain infeasible after applying the common compliance gate (e.g., structurally infeasible regimes under the discrete VM-type pool); $\rho = 2$.

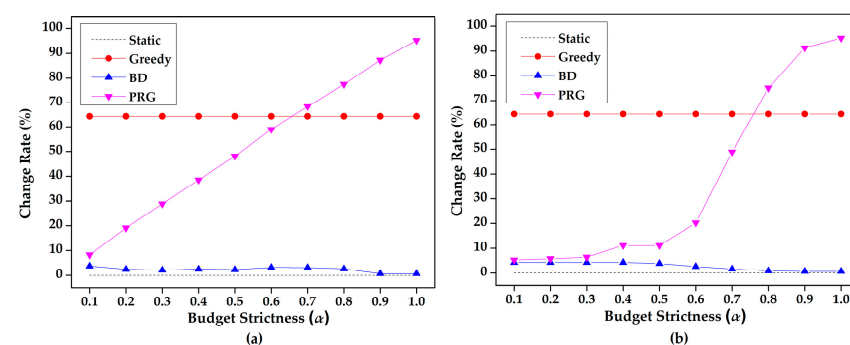


Figure 13. Real-trace operational churn vs. budget strictness (post-gate change rate). Change rate (%) versus α on the Google ClusterData trace under (a) absolute cumulative B_{abs_cum} and (b) per-window run-rate B_{rate_pw} . Change rate is computed from post-gate configurations and reflects deployment-level operational churn (fraction of VMs changing type per window); $\rho = 2$.

Under the run-rate budget model (Figures 11b–13b), BD demonstrates a clear feasibility transition as the budget is relaxed. For example, at $\alpha \approx 0.6$, BD achieves

approximately 39% cost saving while maintaining a very low change rate of about 2–3% and limiting post-gate violations to roughly 5–6% of windows. In contrast, Greedy attains comparable cost savings but exhibits substantially higher operational churn (around 64% change rate) and significantly higher violation rates in the same regime. PRG often achieves zero violations but incurs much higher configuration changes than BD, indicating that feasibility is obtained through aggressive reconfiguration.

Under the absolute cumulative budget model (Figures 11a–13a), feasibility improves more gradually as α increases because the constraint couples spending across time. Nevertheless, BD consistently maintains very low operational churn throughout the sweep and eventually reaches full feasibility at higher α values while still providing meaningful cost savings. In contrast, Greedy achieves feasibility earlier but only at the cost of persistently high reconfiguration rates.

Overall, the real-trace experiment confirms that the qualitative behavior observed in synthetic workloads persists under realistic demand dynamics. In particular, BD maintains a favorable balance between economic efficiency, budget compliance, and operational stability. These results indicate that the proposed dual-based control mechanism generalizes beyond simulator-generated workloads and remains effective under real-world workload variability.

7. Conclusions

This paper investigated FinOps-aware cloud resource management through online VM resizing under temporal hard budgets. In realistic deployments with a discrete VM-type pool, strict budget feasibility cannot be assumed: some decision windows can be structurally infeasible, and candidate assignments must pass a common compliance gate to become deployable actions. Building on these operational constraints, we formulated the Budget-Constrained VM Resizing (BC-VMR) problem, unified multiple temporal budget semantics within a single framework (run-rate, absolute cumulative, incremental cumulative, and rolling), and established NP-hardness for simplified scalarized variants as a complexity-positioning result. Together, these results motivate scalable online methods that explicitly manage violation minimization and stability-aware trade-offs, rather than relying on exact optimization or treating budgets as tunable soft penalties.

Within this framework, we proposed the BD solver, which incorporates temporal hard-budget constraints via an interpretable dual variable ν that acts as a shadow price of budget pressure. The experimental results support the intended behavior of this design: ν decreases monotonically as budgets are relaxed and approaches zero when the corresponding constraint becomes inactive, consistently across budget models. In the main synthetic run-rate evaluation, BD reduces the candidate (pre-gate) budget violation rate to 0.0% once $\alpha \geq 0.6$; after the common compliance gate, any remaining deployment-level violation is confined to structurally infeasible windows. At the same time, BD improves operational stability relative to utility-driven heuristics, reducing resizing-induced churn from 53.95% (Greedy) to 7.80% (BD, $\rho = 2$) in the oscillatory scenario while retaining meaningful cost savings. The comparative analysis further highlights that temporal budget semantics are not interchangeable: different budget models induce distinct feasibility dynamics and stability–efficiency trade-offs, and controllers must be evaluated under a consistent compliance-gate protocol to ensure fair, deployment-aligned comparisons. Finally, scalability experiments show that BD remains practical at large fleet sizes; at $N = 10,000$ it requires 10^2 – 10^3 ms per decision window, whereas NSGA-II requires approximately 10^5 ms.

Most experiments in this study remain simulator-based, although Section 6.5 adds a complementary validation on a public Google trace. Within this scope, we focused

on two core objectives to analyze feasibility and operational stability under hard budget constraints: the objective function f_1 , capturing the waste–overload trade-off, and the stability-oriented objective function f_2 , penalizing reconfiguration frequency. While the overload component already incorporates tail demand and workload-risk amplification signals derived from variability and irregularity (e.g., σ and entropy through ω), our formulation remains intentionally lightweight and simulator-driven and therefore does not capture every operational dimension encountered in production (e.g., explicit SLA/latency objectives, heterogeneous migration costs, or forecast uncertainty). These limitations motivate extensions toward richer objectives and more realistic traces while preserving the deployment-aligned hard budget and common compliance gate evaluation protocol established in this paper.

One such direction is to extend the current lightweight overload-risk term into a more explicit and separately controllable burst-risk component. Certain workloads may appear stable on average while still exhibiting short-lived demand spikes that substantially increase operational risk. In these cases, average-based metrics can underestimate the likelihood of transient overload events. Future work may therefore incorporate burst-risk objectives based on within-window variability, combining statistical measures such as standard deviation, the gap between p95 and mean demand, and entropy-based uncertainty indicators, enabling more conservative resizing decisions for spike-prone workloads.

Another promising direction is to account for workload irregularity and unpredictability. Even when workloads share similar mean and variance, their temporal patterns can differ significantly, leading to distinct operational implications. Periodic fluctuations and irregular demand patterns call for different adaptation strategies, particularly in long-term capacity planning and automated control. Incorporating irregularity-aware penalty objectives linked to entropy thresholds or tail-demand criteria would allow resizing policies to explicitly distinguish between predictable and unpredictable workloads.

Beyond instantaneous overload risk and reconfiguration frequency, future studies may also consider minimizing the temporal volatility of resource usage patterns themselves. Stable and consistent usage trajectories offer practical benefits for monitoring, capacity planning, and operational automation. Volatility-oriented objectives that combine normalized standard deviation and entropy measures could therefore favor solutions with more consistent resource usage when cost efficiency and average performance are comparable.

These extensions can be supported through a multi-objective optimization perspective that systematically explores trade-offs among cost efficiency, stability, burst risk, irregularity, and volatility. Pareto-front analysis may be used to characterize structural trade-offs and identify balanced operating regimes, while objective weights could be extended to adaptive parameters responsive to workload characteristics or budget pressure. In addition, augmenting the current window-based decision structure with prediction-aware mechanisms that exploit historical time-series information may enable more proactive and robust resizing strategies. Together, these directions provide a clear pathway for extending the proposed budget-aware framework toward more realistic and comprehensive cloud operation scenarios.

Funding: This research was supported by the MSIT (Ministry of Science, ICT), Korea, under the National Program for Excellence in SW, supervised by the IITP (Institute for Information communications Technology Planning&Evaluation) in 2026 (2021-0-01440).

Data Availability Statement: The data supporting the findings of this study are available from the corresponding author upon reasonable request.

Conflicts of Interest: The author declares no conflicts of interest.

References

- Mann, Z.Á. Resource Optimization Across the Cloud Stack. *IEEE Trans. Parallel Distrib. Syst.* **2018**, *29*, 169–182. [CrossRef]
- Sayadnavard, M.H.; Toroghi Haghighat, A.; Rahmani, A.M. A Multi-Objective Approach for Energy-Efficient and Reliable Dynamic VM Consolidation in Cloud Data Centers. *Eng. Sci. Technol. Int. J.* **2022**, *26*, 100995. [CrossRef]
- FinOps Foundation. FinOps Framework. Available online: <https://www.finops.org/framework/> (accessed on 10 March 2026).
- Amazon Web Services. AWS Budgets. Available online: <https://aws.amazon.com/aws-cost-management/aws-budgets/> (accessed on 10 March 2026).
- Ren, R.; Tang, X.; Li, Y.; Cai, W. Competitiveness of Dynamic Bin Packing for Online Cloud Server Allocation. *IEEE/ACM Trans. Netw.* **2017**, *25*, 1324–1331. [CrossRef]
- Guo, W.; Tian, W.; Ye, Y.; Xu, L.; Wu, K. Cloud Resource Scheduling with Deep Reinforcement Learning and Imitation Learning. *IEEE Internet Things J.* **2021**, *8*, 3576–3586. [CrossRef]
- Osypanka, P.; Nawrocki, P. Resource Usage Cost Optimization in Cloud Computing Using Machine Learning. *IEEE Trans. Cloud Comput.* **2022**, *10*, 2079–2089. [CrossRef]
- Yu, L.; Chen, L.; Cai, Z.; Shen, H.; Liang, Y.; Pan, Y. Stochastic Load Balancing for Virtual Resource Management in Datacenters. *IEEE Trans. Cloud Comput.* **2020**, *8*, 459–472. [CrossRef]
- Mishra, S.K.; Sahoo, B.; Parida, P.P. Load Balancing in Cloud Computing: A Big Picture. *J. King Saud Univ. Comput. Inf. Sci.* **2020**, *32*, 149–158. [CrossRef]
- Alyahya, K.; Rowe, J.E. Landscape Analysis of a Class of NP-Hard Binary Packing Problems. *Evol. Comput.* **2019**, *27*, 47–73. [CrossRef]
- Moges, F.; Abebe, S. Energy-Aware VM Placement Algorithms for the OpenStack Neat Consolidation Framework. *J. Cloud Comput.* **2019**, *8*, 2. [CrossRef]
- Ibrahim, A.; Noshay, M.; Ali, H.A.; Badawy, M. PAPSO: A Power-Aware VM Placement Technique Based on Particle Swarm Optimization. *IEEE Access* **2020**, *8*, 81747–81764. [CrossRef]
- Shabeera, T.P.; Madhu Kumar, S.D.; Salam, S.M.; Murali Krishnan, K. Optimizing VM Allocation and Data Placement for Data-Intensive Applications in Cloud Using ACO Metaheuristic Algorithm. *Eng. Sci. Technol. Int. J.* **2017**, *20*, 616–628. [CrossRef]
- Liu, Y.; Gao, C.; Zhang, Z.; Lu, Y.; Chen, S.; Liang, M.; Tao, L. Solving NP-Hard Problems with Physarum-Based Ant Colony System. *IEEE/ACM Trans. Comput. Biol. Bioinform.* **2017**, *14*, 108–120. [CrossRef]
- Zhang, H.; Liu, W.; Zhang, Z.; Lu, W.; Xie, J. Joint Target Assignment and Power Allocation in Multiple Distributed MIMO Radar Networks. *IEEE Syst. J.* **2021**, *15*, 694–704. [CrossRef]
- Taillandier, F.; Fernandez, C.; Ndiaye, A. Real Estate Property Maintenance Optimization Based on Multiobjective Multidimensional Knapsack Problem. *Comput.-Aided Civ. Infrastruct. Eng.* **2017**, *32*, 227–251. [CrossRef]
- Corus, D.; Oliveto, P.S.; Yazdani, D. Fast Immune System-Inspired Hypermutation Operators for Combinatorial Optimization. *IEEE Trans. Evol. Comput.* **2021**, *25*, 956–970. [CrossRef]
- Cai, W.; Chan, H.C.B.; Wang, X.; Leung, V.C.M. Cognitive Resource Optimization for the Decomposed Cloud Gaming Platform. *IEEE Trans. Circuits Syst. Video Technol.* **2015**, *25*, 2038–2051. [CrossRef]
- Xiao, H.; Hu, Z.; Li, K. Multi-Objective VM Consolidation Based on Thresholds and Ant Colony System in Cloud Computing. *IEEE Access* **2019**, *7*, 53441–53453. [CrossRef]
- Nazir, J.; Iqbal, M.W.; Alyas, T.; Hamid, M.; Saleem, M.; Malik, S.; Tabassum, N. Load Balancing Framework for Cross-Region Tasks in Cloud Computing. *Comput. Mater. Contin.* **2022**, *70*, 1479–1490. [CrossRef]
- Choi, Y.; Lim, Y. Optimization Approach for Resource Allocation on Cloud Computing for IoT. *Int. J. Distrib. Sens. Netw.* **2016**, *12*, 3479247. [CrossRef]
- Zhang, X.; Wu, C.; Li, Z.; Lau, F.C.M. A Truthful $(1 - \epsilon)$ -Optimal Mechanism for On-Demand Cloud Resource Provisioning. *IEEE Trans. Cloud Comput.* **2020**, *8*, 735–748. [CrossRef]
- Li, Y.; Zhao, C.; Tang, X.; Cai, W.; Liu, X.; Wang, G.; Gong, X. Towards Minimizing Resource Usage with QoS Guarantee in Cloud Gaming. *IEEE Trans. Parallel Distrib. Syst.* **2021**, *32*, 426–440.
- Kieffer, E.; Danoy, G.; Brust, M.R.; Bouvry, P.; Nagih, A. Tackling Large-Scale and Combinatorial Bi-Level Problems with a Genetic Programming Hyper-Heuristic. *IEEE Trans. Evol. Comput.* **2020**, *24*, 44–56. [CrossRef]
- Solozabal, R.; Ceberio, J.; Sanchoyerto, A.; Zabala, L.; Blanco, B.; Liberal, F. Virtual Network Function Placement Optimization with Deep Reinforcement Learning. *IEEE J. Sel. Areas Commun.* **2020**, *38*, 292–303.
- Jafarnejad Ghomi, E.; Rahmani, A.M.; Qader, N.N. Service Load Balancing, Scheduling, and Logistics Optimization in Cloud Manufacturing by Using Genetic Algorithm. *Concurr. Comput. Pract. Exp.* **2019**, *31*, e5329. [CrossRef]
- Gamsiz, M.; Özer, A.H. An Energy-Aware Combinatorial Virtual Machine Allocation and Placement Model for Green Cloud Computing. *IEEE Access* **2021**, *9*, 18625–18648. [CrossRef]
- Wan, B.; Dang, J.; Li, Z.; Gong, H.; Zhang, F.; Oh, S. Modeling Analysis and Cost-Performance Ratio Optimization of Virtual Machine Scheduling in Cloud Computing. *IEEE Trans. Parallel Distrib. Syst.* **2020**, *31*, 1518–1532. [CrossRef]

29. Sardaraz, M.; Tahir, M. A Parallel Multi-Objective Genetic Algorithm for Scheduling Scientific Workflows in Cloud Computing. *Int. J. Distrib. Sens. Netw.* **2020**, *16*, 1550147720949142. [[CrossRef](#)]
30. Zuo, L.; Shu, L.; Dong, S.; Zhu, C.; Hara, T. A Multi-Objective Optimization Scheduling Method Based on the Ant Colony Algorithm in Cloud Computing. *IEEE Access* **2015**, *3*, 2687–2699. [[CrossRef](#)]
31. Shrimali, B.; Patel, H. Multi-Objective Optimization Oriented Policy for Performance and Energy Efficient Resource Allocation in Cloud Environment. *J. King Saud Univ. Comput. Inf. Sci.* **2020**, *32*, 860–869. [[CrossRef](#)]
32. Konjaang, J.K.; Xu, L. Multi-Objective Workflow Optimization Strategy (MOWOS) for Cloud Computing. *J. Cloud Comput.* **2021**, *10*, 11. [[CrossRef](#)]
33. Yousefipour, A.; Rahmani, A.M.; Jahanshahi, M. Energy and Cost-Aware Virtual Machine Consolidation in Cloud Computing. *Softw. Pract. Exp.* **2018**, *48*, 1758–1774.
34. Kim, I.K.; Wang, W.; Qi, Y.; Humphrey, M. Forecasting Cloud Application Workloads with CloudInsight for Predictive Resource Management. *IEEE Trans. Cloud Comput.* **2022**, *10*, 1848–1863. [[CrossRef](#)]
35. Cohen, M.C.; Keller, P.W.; Mirrokni, V.; Zadimoghaddam, M. Overcommitment in Cloud Services: Bin Packing with Chance Constraints. *Manag. Sci.* **2019**, *65*, 3255–3271. [[CrossRef](#)]
36. Wu, Q.; Ishikawa, F.; Zhu, Q.; Xia, Y.; Wen, J. Deadline-Constrained Cost Optimization Approaches for Workflow Scheduling in Clouds. *IEEE Trans. Parallel Distrib. Syst.* **2017**, *28*, 3401–3412. [[CrossRef](#)]
37. Duque, R.; Arbelaez, A.; Díaz, J.F. Online Over Time Processing of Combinatorial Problems. *Constraints* **2018**, *23*, 310–334. [[CrossRef](#)]
38. Thanasias, V.; Lee, C.; Hanif, M.; Kim, E.; Helal, S. VM Capacity-Aware Scheduling within Budget Constraints in IaaS Clouds. *PLoS ONE* **2016**, *11*, e0160456. [[CrossRef](#)]
39. Rizvi, N.; Ramesh, D. HBDCWS: Heuristic-Based Budget and Deadline Constrained Workflow Scheduling Approach for Heterogeneous Clouds. *Soft Comput.* **2020**, *24*, 18971–18990. [[CrossRef](#)]
40. Rajasekar, P.; Santhiya, P. Budget-Based Resource Provisioning and Scheduling Algorithm for Scientific Workflows on IaaS Cloud. *Multimed. Tools Appl.* **2024**, *83*, 50981–51007.
41. Radhika, E.G.; Sadasivam, G.S. Budget Optimized Dynamic Virtual Machine Provisioning in Hybrid Cloud Using Fuzzy Analytic Hierarchy Process. *Expert Syst. Appl.* **2021**, *183*, 115398. [[CrossRef](#)]
42. Arabnejad, H.; Barbosa, J.G. A Budget Constrained Scheduling Algorithm for Workflow Applications. *J. Grid Comput.* **2014**, *12*, 665–679. [[CrossRef](#)]
43. Xiao, L.; Xiao, Z.; Wu, D.; Hu, M.; Zhou, Y. CRS: A Cost-Aware Resource Scheduling Framework for Deep Learning Task Orchestration in Mobile Clouds. *IEEE Trans. Mob. Comput.* **2025**, *24*, 600–613.
44. Bandapati, G. FinOps-Driven Strategies for Large-Scale Cloud Cost Optimization. *Int. J. Intell. Syst. Appl. Eng.* **2024**, *12*, 2203–2209.
45. Google Cluster-Usage Traces. Available online: <https://github.com/google/cluster-data/> (accessed on 10 March 2026).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.